

第 8 章

排 序

【考纲内容】

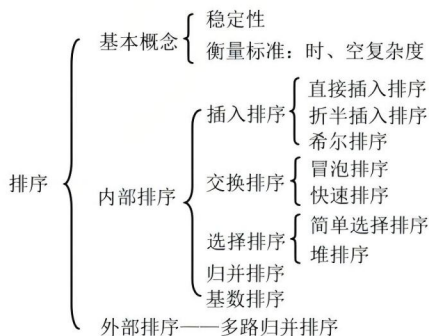
- (一) 排序的基本概念
- (二) 插入排序
 - 直接插入排序；折半插入排序；希尔排序 (shell sort)
- (三) 交换排序
 - 冒泡排序 (bubble sort)；快速排序
- (四) 选择排序
 - 简单选择排序；堆排序
- (五) 2 路归并排序 (merge sort)
- (六) 基数排序
- (七) 外部排序
- (八) 排序算法的分析和应用

扫一扫



视频讲解

【知识框架】



【复习提示】

堆排序、快速排序和归并排序是本章的重点与难点。读者应深入掌握各类算法的思想、排序过程（能动手模拟）及其特性，包括初始状态的影响、时空复杂度、稳定性和适用场景等。考试通常以选择题形式考查不同算法的对比。此外，应对常用排序算法的关键代码做到熟练编写，并能根据给定序列的特点，结合算法特性选择最合适的排序方法。

8.1 排序的基本概念

8.1.1 排序的定义

排序是指将表中的元素重新排列，使其按关键字有序的过程。在许多应用中，为了便于后续查找，通常会将表组织成有序形式。排序的确切定义如下。

输入: n 个记录 $R_1, R_2, \dots, R_n$, 对应的关键字为 $k_1, k_2, \dots, k_n$ 。

输出: 输入序列的一个重排 $R'_1, R'_2, \dots, R'_n$, 使得 $k'_1 \leq k'_2 \leq \dots \leq k'_n$ (其中“ $\leq$ ”可根据需要替换为其他比较关系)。

算法的稳定性。若待排序表中有两个元素 R_i 和 R_j , 则其关键字相同 ($\text{key}_i = \text{key}_j$), 且排序前 R_i 位于 R_j 之前; 若使用某一排序算法排序后, R_i 仍位于 R_j 之前, 则称该排序算法是稳定的, 否则, 称为不稳定的。需要注意的是, 稳定性并不能用来衡量算法的优劣, 而只是描述其性质之一。当待排序关键字互不重复时, 排序结果唯一, 此时算法是否稳定就无关紧要了。

注意

对于不稳定的排序算法, 只需举出一组关键字的实例, 证明它的不稳定性即可。

在排序过程中, 根据数据是否全部存放在内存中, 可将排序算法分为两类: ① 内部排序, 指排序期间所有元素都存放在内存中; ② 外部排序, 指排序期间元素无法全部装入内存, 需要在排序过程中频繁地在内、外存之间交换数据。

一般情况下, 内部排序算法在执行过程中主要涉及两种基本操作: 比较和移动。通过比较关键字的大小来确定元素的相对顺序, 并通过移动元素使整个序列达到有序状态。当然, 并非所有内部排序都基于比较操作——例如, 基数排序就是一种典型的非比较排序。

各类排序算法各有优缺点, 适用于不同场景。通常可将排序算法分为五大类: 插入排序、交换排序、选择排序、归并排序和基数排序, 后续几节将分别介绍。内部排序算法的性能主要由其时间复杂度和空间复杂度决定, 而时间复杂度通常取决于比较和移动操作的次数。

注意

大多数内部排序算法更适用于顺序存储的线性表。

8.1.2 本节试题精选

单项选择题

01. 下述排序算法中, 不属于内部排序算法的是 ()。
A. 插入排序 B. 选择排序 C. 拓扑排序 D. 冒泡排序
02. 排序算法的稳定性是指 ()。
A. 经过排序后, 能使关键字相同的元素保持原顺序中的相对位置不变
B. 经过排序后, 能使关键字相同的元素保持原顺序中的绝对位置不变
C. 排序算法的性能与被排序元素个数关系不大
D. 排序算法的性能与被排序元素的个数关系密切
03. 下列关于排序的叙述中, 正确的是 ()。
A. 稳定的排序算法优于不稳定的排序算法
B. 对同一线性表使用不同的排序算法进行排序, 得到的排序结果可能不同
C. 排序算法都是在顺序表上实现的, 在链表上无法实现排序算法
D. 在顺序表上实现的排序算法在链表上也可以实现

8.1.3 答案与解析

单项选择题

01. C

拓扑排序是将有向图中所有结点排成一个线性序列, 虽然也是在内存中进行的, 但它不属于

我们这里所提到的内部排序范畴，也不满足前面排序的定义。

02. A

注意，这里的绝对位置是指若在排序前元素 R 在位置 i ，则绝对位置就是 i ，即排序后 R 的位置不发生变化，显然选项 B 是不对的。选项 C、D 与题目要求无关。

03. B

算法的稳定性与算法优劣无关，选项 A 排除。使用链表也可以进行排序，只是有些排序算法不再适用，因为这时定位元素只能顺序逐链查找，如折半插入排序。

8.2 插入排序

插入排序是一种简单直观的排序算法，其基本思想是：每次将一个待排序的记录，按其关键字大小插入到前面已经排好序的子序列中，直到所有记录都插入完毕。基于这一思想，可以引申出三种重要的排序算法：直接插入排序、折半插入排序和希尔排序。

8.2.1 直接插入排序^①

根据上述插入排序的思想，不难得出一种最简单也最直观的直接插入排序算法。假设在排序过程中，待排序表 $L[1..n]$ 在某一时刻的状态如下：

有序序列 $L[1..i-1]$	$L(i)$	无序序列 $L[i+1..n]$
------------------	--------	------------------

要将元素 $L(i)$ 插入到已有序的子序列 $L[1..i-1]$ 中，需执行以下操作（为避免混淆，下文用 $L[]$ 表示一个表，而用 $L()$ 表示一个元素）：

- 1) 查找 $L(i)$ 在 $L[1..i-1]$ 中的插入位置 k 。
- 2) 将 $L[k..i-1]$ 中的所有元素依次后移一个位置。
- 3) 将 $L(i)$ 放入位置 k 。

为对整个表 $L[1..n]$ 的排序，可从 $L(2)$ 开始，依次将 $L(2)$ 到 $L(n)$ 插入其前面已排好序的子序列中。初始时， $L[1]$ 可视为一个长度为 1 的有序子序列。上述过程共执行 $n-1$ 次，最终得到一个有序表。直接插入排序通常采用原地排序（空间复杂度为 $O(1)$ ）。在从后往前的比较过程中，需要将已排序的元素逐个后移，为新元素腾出插入位置。

以下是直接插入排序的实现，其中再次使用了前面提到的“哨兵”（作用相同）。

```
void InsertSort(ElemType A[], int n){
    int i, j;
    for(i=2; i<=n; i++){
        if(A[i]<A[i-1]){
            A[0]=A[i];
            for(j=i-1; A[0]<A[j]; --j){
                A[j+1]=A[j];
            }
            A[j+1]=A[0];
        }
    }
}
```

假定初始序列为 49, 38, 65, 97, 76, 13, 27, 49，初始时 49 可视为一个已排好序的子序列。按

^① 在本书中，凡是没有特殊注明的，通常默认排序结果为非递减有序序列。

考点追踪 ▶ 直接插入排序和折半插入排序的比较 (2012)

空间效率: 与直接插入排序相同, 仅使用常数个辅助单元, 空间复杂度为 $O(1)$ 。

时间效率: 折半插入排序的主要改进在于减少了关键字的比较次数。由于采用折半查找, 比较次数约为 $O(n\log_2 n)$, 仅取决于元素个数 n 。然而, 元素的移动次数并未减少, 仍然依赖于初始序列的有序程度。因此, 折半插入排序的时间复杂度仍为 $O(n^2)$ 。

稳定性: 在插入过程中保持了相同关键字元素的相对顺序, 因此是稳定的排序算法。

适用性: 该算法仅适用于顺序存储的线性表。

8.2.3 希尔排序

由前文分析可知, 直接插入排序的时间复杂度为 $O(n^2)$, 但在待排序列接近正序时, 其时间效率可提升至 $O(n)$ 。因此, 该算法更适用于基本有序或数据量较小的排序场景。希尔排序正是基于这一特性对直接插入排序进行改进, 也称为缩小增量排序。

考点追踪 ▶ 希尔排序的特性与分析 (2014、2015、2018、2025)

希尔排序的基本思想是: 通过逐步缩小增量的方式, 对表中相隔一定距离的元素构成的子序列进行多次直接插入排序, 使整个表逐渐趋于基本有序, 最终通过一趟完整的直接插入排序完成全局排序。具体而言, 算法首先选取一个小于 n 的初始增量 d_1 , 将待排序表划分为 d_1 个子序列 (每个子序列包含下标间隔为 d_1 的记录), 并对各子序列分别进行直接插入排序; 随后依次取更小的增量 $d_2, d_3, \dots$ (满足 $d_1 > d_2 > \dots > d_i = 1$), 重复分组与排序操作。当增量减至 1 时, 所有记录属于同一子序列, 且序列已高度有序, 此时执行最后一趟直接插入排序, 即可快速获得最终结果。

注意, 目前尚未找到理论上最优的增量序列。仍以 8.2.1 节的关键字为例, 假定第一趟取 $d_1 = 5$, 形成 5 个子序列 (对应图中第 2 至第 6 行), 排序后结果如第 7 行所示; 第二趟取 $d_2 = 3$, 对 3 个子序列排序, 结果见第 11 行; 最后对整个序列进行一趟排序, 完整过程如图 8.2 所示。

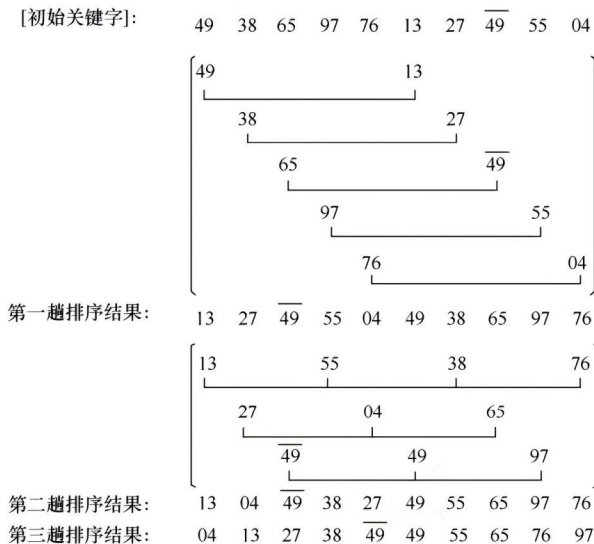


图 8.2 希尔排序示例

希尔排序的算法实现如下:

```
void ShellSort(ElemType A[],int n){
    //A[0]只是暂存单元,不是哨兵,当j<=0时,插入位置已到
    int dk,i,j;
```



```

for (dk=n/2; dk>=1; dk=dk/2)           //增量变化 (无统一规定)
    for (i=dk+1; i<=n; ++i)
        if (A[i]<A[i-dk]) {               //需将 A[i] 插入有序增量子表
            A[0]=A[i];                     //暂存在 A[0]
            for (j=i-dk; j>0&&A[0]<A[j]; j-=dk)
                A[j+dk]=A[j];             //记录后移, 查找插入的位置
            A[j+dk]=A[0];                 //插入
        }
    }

```

希尔排序算法的性能分析如下:

空间效率: 仅使用常数个辅助单元, 空间复杂度为 $O(1)$ 。

时间效率: 时间复杂度依赖于所选增量序列, 而最优增量序列仍是数学上的未解难题, 因此精确分析较为困难。当 n 在常见范围内时, 时间复杂度约为 $O(n^{1.3})$; 最坏情况下退化为 $O(n^2)$ 。

稳定性: 当相同关键字的记录被划分到不同子表时, 可能会改变它们的相对次序, 因此希尔排序是不稳定的排序算法。例如, 图 8.2 中 49 与 $\overline{49}$ 在排序过程中相对顺序发生了变化。

适用性: 希尔排序仅适用于顺序存储的线性表。

8.2.4 本节试题精选

一、单项选择题

01. 对 5 个不同的数据元素进行直接插入排序, 最多需要进行的比较次数是 ()。注意, 哨兵的比较不计入次数。
A. 8 B. 10 C. 15 D. 25
02. 在待排序的元素序列基本有序的前提下, 效率最高的排序算法是 ()。
A. 直接插入排序 B. 简单选择排序 C. 快速排序 D. 归并排序
03. 对有 n 个元素的顺序表采用直接插入排序算法进行排序, 在最坏情况下所需的比较次数是 (), 在最好情况下所需的比较次数是 ()。
A. $n-1$ B. $n+1$ C. $n/2$ D. $n(n-1)/2$
04. 数据序列 {8, 10, 13, 4, 6, 7, 22, 2, 3} 只能是 () 两趟排序后的结果。
A. 简单选择排序 B. 冒泡排序 C. 直接插入排序 D. 堆排序
05. 用直接插入排序算法对下列 4 个表进行 (从小到大) 排序, 比较次数最少的是 ()。
A. 94, 32, 40, 90, 80, 46, 21, 69 B. 21, 32, 46, 40, 80, 69, 90, 94
C. 32, 40, 21, 46, 69, 94, 90, 80 D. 90, 69, 80, 46, 21, 32, 94, 40
06. 在下列算法中, () 算法可能出现下列情况: 在最后一趟开始之前, 所有元素都不在最终位置上。
A. 堆排序 B. 冒泡排序 C. 直接插入排序 D. 快速排序
07. 希尔排序属于 ()。
A. 插入排序 B. 交换排序 C. 选择排序 D. 归并排序
08. 对序列 {15, 9, 7, 8, 20, -1, 4} 采用希尔排序, 经一趟后序列变为 {15, -1, 4, 8, 20, 9, 7}, 则该次采用的增量是 ()。
A. 1 B. 4 C. 3 D. 2
09. 若序列 {15, 9, 7, 8, 20, -1, 4} 经一趟排序后变成 {9, 15, 7, 8, 20, -1, 4}, 则采用的是 () 方法。
A. 选择排序 B. 快速排序 C. 直接插入排序 D. 冒泡排序

10. 对序列{98, 36, -9, 0, 47, 23, 1, 8, 10, 7}采用希尔排序, 下列序列()是增量为4的一趟排序结果。
A. {10, 7, -9, 0, 47, 23, 1, 8, 98, 36} B. {-9, 0, 36, 98, 1, 8, 23, 47, 7, 10}
C. {36, 98, -9, 0, 23, 47, 1, 8, 7, 10} D. 以上都不对
11. 对序列{E, A, S, Y, Q, U, E, S, T, I, O, N}按照字典顺序排序, 采用增量 $d=6, 3, 1$ 的希尔排序算法。则前两趟排序后, 关键字的总比较次数为()。
A. 15 B. 17 C. 16 D. 18
12. 已知输入序列{13, 24, 7, 1, 8, 9, 11, 56, 34, 51, 2, 77, 5}, 增量序列 $d=5, 3, 1$, 采用希尔排序算法进行排序, 则两趟排序后的结果为()。
A. 1, 7, 8, 9, 13, 24, 11, 34, 51, 2, 5, 56, 77
B. 1, 7, 5, 2, 8, 9, 24, 11, 34, 51, 13, 77, 56
C. 2, 11, 5, 1, 8, 9, 24, 7, 34, 51, 13, 77, 56
D. 2, 5, 11, 1, 8, 9, 7, 24, 34, 13, 51, 77, 56
13. 折半插入排序算法的时间复杂度为()。
A. $O(n)$ B. $O(n\log_2 n)$ C. $O(n^2)$ D. $O(n^3)$
14. 有些排序算法在每趟排序过程中, 都会有一个元素被放置到其最终位置上, ()算法不一定会出现此种情况。
A. 希尔排序 B. 堆排序 C. 冒泡排序 D. 快速排序
15. 以下排序算法中, 不稳定的是()。
A. 冒泡排序 B. 直接插入排序
C. 希尔排序 D. 归并排序
16. 以下排序算法中, 稳定的是()。
A. 快速排序 B. 堆排序 C. 直接插入排序 D. 简单选择排序
17. 【2012 统考真题】对同一待排序序列分别进行折半插入排序和直接插入排序, 两者之间可能的不同之处是()。
A. 排序的总趟数 B. 元素的移动次数
C. 使用辅助空间的数量 D. 元素之间的比较次数
18. 【2014 统考真题】用希尔排序算法对一个数据序列进行排序时, 若第一趟排序结果为9, 1, 4, 13, 7, 8, 20, 23, 15, 则该趟排序采用的增量(间隔)可能是()。
A. 2 B. 3 C. 4 D. 5
19. 【2015 统考真题】希尔排序的组内排序采用的是()。
A. 直接插入排序 B. 折半插入排序 C. 快速排序 D. 归并排序
20. 【2018 统考真题】对初始数据序列(8, 3, 9, 11, 2, 1, 4, 7, 5, 10, 6)进行希尔排序。若第一趟排序结果为(1, 3, 7, 5, 2, 6, 4, 9, 11, 10, 8), 第二趟排序结果为(1, 2, 6, 4, 3, 7, 5, 8, 11, 10, 9), 则两趟排序采用的增量(间隔)依次是()。
A. 3, 1 B. 3, 2 C. 5, 2 D. 5, 3

二、综合应用题

01. 给出关键字序列{4, 5, 1, 2, 6, 3}的直接插入排序过程。
02. 给出关键字序列{50, 26, 38, 80, 70, 90, 8, 30, 40, 20}的希尔排序过程(取增量序列为 $d=\{5, 3, 1\}$, 排序结果为从小到大排列)。

8.2.5 答案与解析

一、单项选择题

01. B

直接插入排序在最坏的情况下要做 $n(n-1)/2$ 次关键字的比较, 当 $n=5$ 时, 关键字的比较次数为 10。注意不考虑与哨兵的比较。

02. A

序列初始基本有序, 因此使用直接插入排序算法的时间复杂度接近 $O(n)$, 而使用其他算法的时间复杂度均大于 $O(n)$ 。

03. D、A

待排序表为反序时, 直接插入排序需要进行 $n(n-1)/2$ 次比较(从前往后依次需要比较 1, 2, ..., $n-1$ 次); 待排序表为正序时, 只需进行 $n-1$ 次比较。注意本题不考虑与哨兵的比较。

04. C

冒泡排序和选择排序经过两趟排序后, 应该有两个最大(或最小)元素放在其最终位置; 插入排序经过两趟排序后, 前三个元素应该是局部有序的。只可能是插入排序。

注 意

在排序过程中, 每趟都能确定一个元素在其最终位置的有冒泡排序、简单选择排序、堆排序、快速排序, 其中前三者能形成全局有序的子序列, 后者能确定枢轴元素的最终位置。

05. B

越接近正序的序列, 直接插入排序的比较次数就越少。选项 B 和 C 是比较接近正序的, 然后分别判断两个序列的比较次数, 以选项 B 为例: 第一趟, 插入 32, 比较 1 次; 第二趟, 插入 46, 比较 1 次; 第三趟, 插入 40, 因为 40 比 46 小但比 32 大, 所以比较 2 次; 第四趟, 插入 80, 比较 1 次; 第五趟, 插入 69, 比较 2 次; 以此类推, 共比较 9 次。同理求出选项 C 的比较次数为 11 次。所以选择选项 B。

06. C

在直接插入排序中, 若待排序列中的最后一个元素应插入表中的第一个位置, 则前面的有序子序列中的所有元素都不在最终位置上。

07. A

希尔排序是对直接插入排序算法改进后提出来的, 本质上仍属于插入排序的范畴。

08. B

希尔排序将序列分成若干组, 记录只在组内进行交换。由观察可知, 经过一趟后 9 和 -1 交换, 7 和 4 交换, 可知增量为 4。

09. C

前两个元素已经局部有序, 很明显一趟直接插入排序算法有效。再排除其他算法即可。

10. A

增量为 4 意味着所有相距为 4 的记录构成一组, 然后在组内进行直接插入排序, 经观察, 只有选项 A 满足要求。

11. B

第一趟: EE 为一组, 比较; AS 为一组, 比较; ST 为一组, 比较; YI 为一组, 比较后交换; QO 为一组, 比较后交换; UN 为一组, 比较后交换, 结果为 EASIONESTYQU。第二趟: EIEY 为一组, 用直接插入排序需要依次比较 I 和 E、E 和 I、E 和 E、Y 和 I; AOSQ 为一组, 依次比较 O 和 A、S

和 O、Q 和 S、Q 和 O；SNTU 为一组，依次比较 N 和 S、T 和 S、U 和 T。第一趟比较次数为 6，第二趟比较次数为 11，总比较次数为 17。

12. B

第一趟增量 $d=5$ ，第一趟排序后，结果为 2, 11, 5, 1, 8, 9, 24, 7, 34, 51, 13, 77, 56。第二趟增量 $d=3$ ，第二趟排序后，结果为 1, 7, 5, 2, 8, 9, 24, 11, 34, 51, 13, 77, 56。

13. C

虽然折半插入排序是对直接插入排序的改进，但它改进的只是比较的次数，而移动次数未发生变化，时间复杂度仍为 $O(n^2)$ 。

14. A

因为希尔排序是基于插入排序算法提出的，所以它不一定在每趟排序过程后将某一元素放置到最终位置上。

15. C

希尔排序是一种复杂的插入排序算法，它是一种不稳定的排序算法。

16. C

基于插入、交换、选择的三类排序算法中，通常简单方法是稳定的（直接插入、折半插入、冒泡），但有一个例外就是简单选择，复杂方法都是不稳定的（希尔排序、快速排序、堆排序）。

17. D

折半插入排序与直接插入排序都将待插入元素插入前面的有序子表，区别是：确定当前记录在前面有序子表中的位置时，直接插入排序采用顺序查找法，而折半插入排序采用折半查找法。排序的总趟数取决于元素个数 n ，两者都是 $n-1$ 趟。元素的移动次数都取决于初始序列，两者相同。使用辅助空间的数量也都是 $O(1)$ 。折半插入排序的比较次数与序列初态无关，时间复杂度为 $O(n\log_2 n)$ ；而直接插入排序的比较次数与序列初态有关，时间复杂度为 $O(n) \sim O(n^2)$ 。

18. B

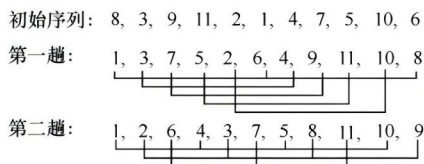
首先，第二个元素为 1，是整个序列中的最小元素，可知该希尔排序为从小到大排序。然后考虑增量问题，若增量为 2，则第 $1+2$ 个元素 4 明显比第 1 个元素 9 要小，排除选项 A。若增量为 3，则第 $i, i+3, i+6$ ($i=1, 2, 3$) 个元素都为有序序列，符合希尔排序的特点。若增量为 4，则第 1 个元素 9 比第 $1+4$ 个元素 7 要大，排除选项 C。若增量为 5，则第 1 个元素 9 比第 $1+5$ 个元素 8 要大，排除选项 D。

19. A

希尔排序的思想是：先将待排元素序列分割成若干子序列（由相隔某个“增量”的元素组成），分别进行直接插入排序，然后依次缩减增量再进行排序，待整个序列中的元素基本有序（增量足够小）时，再对全体元素进行一次直接插入排序。

20. D

如下图所示。



第一趟分组: 8, 1, 6; 3, 4; 9, 7; 11, 5; 2, 10; 间隔为 5，排序后组内递增。

第二趟分组: 1, 5, 4, 10; 3, 2, 9, 8; 7, 6, 11; 间隔为 3，排序后组内递增。

因此，选择选项 D。

二、综合应用题

01. 【解答】

直接插入排序过程如下。

假设划分操作已封装为函数 `Partition()`，其功能是对子表 `A[low...high]` 进行一趟划分，并返回枢轴元素的最终位置 `k`。在此基础上，快速排序的递归实现如下：

```
void QuickSort(ElemType A[],int low,int high){
    if(low<high){ //递归跳出的条件
        //Partition()就是划分操作，将表A[low...high]划分为满足上述条件的两个子表
        int pivotpos=Partition(A,low,high); //划分
        QuickSort(A,low,pivotpos-1); //依次对两个子表进行递归排序
        QuickSort(A,pivotpos+1,high);
    }
}
```

考点追踪 ▶ (算法题) 快速排序中划分操作的应用 (2016)

考研所考查的划分操作，通常以子表的第一个元素作为枢轴来对表进行划分，将比枢轴小的元素逐步移至左侧，比枢轴大的元素移至右侧，最终使枢轴归位。典型实现如下：

```
int Partition(ElemType A[],int low,int high){ //一趟划分
    ElemType pivot=A[low]; //将当前表中第一个元素设为枢轴，对表进行划分
    while(low<high){ //循环跳出条件
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high]; //将比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low]; //将比枢轴大的元素移动到右端
    }
    A[low]=pivot; //枢轴元素存放到最终位置
    return low; //返回存放枢轴的最终位置
}
```

快速排序的性能高度依赖于划分的平衡性，其性能分析如下。

考点追踪 ▶ 快速排序中递归次数的影响因素分析 (2010)

空间效率：由于采用递归实现，算法需借助系统栈保存每层递归的上下文信息，栈的容量等于递归调用的最大层数。最好情况下（每次划分均分）为 $O(\log_2 n)$ ；最坏情况下（划分极不平衡），需进行 $n-1$ 次递归调用，栈的容量为 $O(n)$ ；平均情况下为 $O(\log_2 n)$ 。

时间效率：最坏情况发生在初始序列基本有序或基本逆序时，每趟划分仅减少一个元素，总比较次数约为 $n(n-1)/2$ ，时间复杂度为 $O(n^2)$ 。最好情况是每次划分都将序列等分为两部分，此时时间复杂度为 $O(n\log_2 n)$ 。值得注意的是，快速排序的平均运行时间非常接近其最优情况，远优于最坏情况，因此它是所有内部排序算法中平均性能最优的排序方法。

有很多方法可以提高划分的平衡性：一种是三数取中法，从子序列的首、尾及中间三个位置各取一个元素，以其中位数作为枢轴；或者随机选取子序列中的某个元素作为枢轴。

稳定性：在划分过程中，若右侧区域有两个关键字相等，且均小于枢轴，则交换到左侧区域后，它们的相对位置会发生改变，因此快速排序是不稳定的排序算法。例如，表 $L = \{3, 2, 2\}$ ，经过一趟排序后 $L = \{2, 2, 3\}$ ，也是最终排序结果，显然，2 与 2 的相对次序已发生了变化。

考点追踪 ▶ 快速排序适合采用的存储方式 (2011)

适用性：快速排序要求能够随机访问元素，因此仅适用于顺序存储的线性表。

注 意

快速排序在每趟划分后并不产生全局有序的子序列。然而，每趟划分都能确保当前所选的枢轴元素被精确放置到其在整个序列中的最终位置上。这一特性是快速排序高效性的关键所在。

8.3.3 本节试题精选

一、单项选择题

01. 对 n 个不同的元素利用冒泡法从小到大排序, 在 () 情况下元素交换的次数最多。
A. 从大到小排列好的 B. 从小到大排列好的
C. 元素无序 D. 元素基本有序
02. 若用冒泡排序算法对序列 {10, 14, 26, 29, 41, 52} 从大到小排序, 则需进行 () 次比较。
A. 3 B. 10 C. 15 D. 25
03. 用某种排序算法对线性表 {25, 84, 21, 47, 15, 27, 68, 35, 20} 进行排序时, 元素序列的变化情况如下:
1) 25, 84, 21, 47, 15, 27, 68, 35, 20
2) 20, 15, 21, 25, 47, 27, 68, 35, 84
3) 15, 20, 21, 25, 35, 27, 47, 68, 84
4) 15, 20, 21, 25, 27, 35, 47, 68, 84
则所采用的排序算法是 ()。
A. 选择排序 B. 插入排序 C. 2 路归并排序 D. 快速排序
04. 一组记录的关键码为 (46, 79, 56, 38, 40, 84), 则利用快速排序算法, 以第一个记录为基准, 从小到大得到的一次划分结果为 ()。
A. (38, 40, 46, 56, 79, 84) B. (40, 38, 46, 79, 56, 84)
C. (40, 38, 46, 56, 79, 84) D. (40, 38, 46, 84, 56, 79)
05. 快速排序算法在 () 情况下最不利于发挥其长处。
A. 要排序的数据量太大 B. 要排序的数据中含有多个相同值
C. 要排序的数据个数为奇数 D. 要排序的数据已基本有序
06. 就平均性能而言, 目前最好的内部排序算法是 ()。
A. 冒泡排序 B. 直接插入排序 C. 希尔排序 D. 快速排序
07. 数据序列 $F = \{2, 1, 4, 9, 8, 10, 6, 20\}$ 只能是下列排序算法中的 () 两趟排序后的结果。
A. 快速排序 B. 冒泡排序 C. 选择排序 D. 插入排序
08. 对元素序列 {8, 9, 10, 4, 5, 6, 20, 1, 2} 采用冒泡排序 (从后往前次序进行, 要求升序), 需要进行元素交换的趟数至少是 () (不考虑无元素交换的最后一趟)。
A. 3 B. 4 C. 5 D. 8
09. 双向冒泡排序是指对一个序列在正反两个方向交替进行扫描, 第一趟把最大值放在序列的最右端, 第二趟把最小值放在序列的最左端, 之后在缩小的范围内进行同样的扫描, 放在次右端、次左端, 直至序列有序。对数组 {4, 7, 8, 3, 5, 6, 10, 9, 1, 2} 进行双向冒泡排序, 则排序趟数是 ()。(第一趟从左往右开始, 从左往右或从右往左都称为一趟)。
A. 7 B. 6 C. 8 D. 9
10. 对下列关键字序列用快速排序进行排序时, 每次选取的基准元素都为待处理序列的第一个元素, 速度最快的情形是 (), 速度最慢的情形是 ()。
A. {21, 25, 5, 17, 9, 23, 30} B. {25, 23, 30, 17, 21, 5, 9}
C. {21, 9, 17, 30, 25, 23, 5} D. {5, 9, 17, 21, 23, 25, 30}
11. 对下列 4 个序列, 以第一个关键字为基准用快速排序算法进行排序, 在第一趟过程中移动记录次数最多的是 ()。

- A. 92, 96, 88, 42, 30, 35, 110, 100 B. 92, 96, 100, 110, 42, 35, 30, 88
C. 100, 96, 92, 35, 30, 110, 88, 42 D. 42, 30, 35, 92, 100, 96, 88, 110
12. 下列序列中, () 可能是执行第一趟快速排序后所得到的序列 (按从大到小排序和从小到大排序来分别讨论)。
I. {68, 11, 18, 69, 23, 93, 73} II. {68, 11, 69, 23, 18, 93, 73}
III. {93, 73, 68, 11, 69, 23, 18} IV. {68, 11, 69, 23, 18, 73, 93}
A. I、IV B. II、III C. III、IV D. 只有 IV
13. 对 n 个关键字进行快速排序, 最大递归深度为 (), 最小递归深度约为 ()。
A. 1 B. n C. $\log_2 n$ D. $n \log_2 n$
14. 对 8 个元素的序列进行快速排序, 在最好情况下的关键字比较次数是 ()。
A. 7 B. 8 C. 12 D. 13
15. 【2010 统考真题】采用递归方式对顺序表进行快速排序。下列关于递归次数的叙述中, 正确的是 ()。
A. 递归次数与初始数据的排列次序无关
B. 每次划分后, 先处理较长的分区可以减少递归次数
C. 每次划分后, 先处理较短的分区可以减少递归次数
D. 递归次数与每次划分后得到的分区的处理顺序无关
16. 【2011 统考真题】为实现快速排序算法, 待排序序列宜采用的存储方式是 ()。
A. 顺序存储 B. 散列存储 C. 链式存储 D. 索引存储
17. 【2014 统考真题】下列选项中, 不可能是快速排序第二趟排序结果的是 ()。
A. 2, 3, 5, 4, 6, 7, 9 B. 2, 7, 5, 6, 4, 3, 9
C. 3, 2, 5, 4, 7, 6, 9 D. 4, 2, 3, 5, 7, 6, 9
18. 【2019 统考真题】排序过程中, 对尚未确定最终位置的所有元素进行一遍处理称为一趟。下列序列中, 不可能是快速排序第二趟结果的是 ()。
A. 5, 2, 16, 12, 28, 60, 32, 72 B. 2, 16, 5, 28, 12, 60, 32, 72
C. 2, 12, 16, 5, 28, 32, 72, 60 D. 5, 2, 12, 28, 16, 32, 72, 60
19. 【2023 统考真题】使用快速排序算法对数据进行升序排序, 若经过一次划分后得到的数据序列是 68, 11, 70, 23, 80, 77, 48, 81, 93, 88, 则该次划分的枢轴是 ()。
A. 11 B. 70 C. 80 D. 81
20. 【2024 统考真题】使用快速排序算法对含 n ($n \geq 3$) 个元素的数组 M 进行排序, 若第一趟排序将 M 中除枢轴外的 $n-1$ 个元素划分为均不为空的 P 和 Q 两块, 则下列叙述中, 正确的是 ()。
A. P 与 Q 块间有序 B. P 与 Q 均块内有序
C. P 和 Q 的元素个数大致相等 D. P 中和 Q 中均不存在相等的元素

二、综合应用题

01. 已知线性表按顺序存储, 且每个元素都是不相同的整数型元素, 设计把所有奇数移动到所有偶数前边的算法 (要求时间最短, 辅助空间最小)。
02. 试编写一个算法, 使之能够在数组 $L[1..n]$ 中找出第 k 小的元素 (从小到大排序后处于第 k 个位置的元素)。
03. 荷兰国旗问题: 设有一个仅由红、白、蓝三种颜色的条块组成的条块序列, 存储在

一个顺序表中, 请编写一个时间复杂度为 $O(n)$ 的算法, 使得这些条块按红、白、蓝的顺序排好, 即排成荷兰国旗图案。请完成算法实现:

```
typedef enum{RED, WHITE, BLUE} color;      //设置枚举数组
void Flag_Arrange(color a[],int n){ ... }
```

04. 【2016 统考真题】已知由 n ($n \geq 2$) 个正整数构成的集合 $A = \{a_k | 0 \leq k < n\}$, 将其划分为两个不相交的子集 A_1 和 A_2 , 元素个数分别是 n_1 和 n_2 , A_1 和 A_2 中的元素之和分别为 S_1 和 S_2 。设计一个尽可能高效的划分算法, 满足 $|n_1 - n_2|$ 最小且 $|S_1 - S_2|$ 最大。要求:
- 1) 给出算法的基本设计思想。
 - 2) 根据设计思想, 采用 C 或 C++ 语言描述算法, 关键之处给出注释。
 - 3) 说明所设计算法的平均时间复杂度和空间复杂度。

8.3.4 答案与解析

一、单项选择题

01. A

冒泡排序最少进行 1 趟冒泡, 最多进行 $n-1$ 趟冒泡。初始序列为逆序时, 需进行 $n-1$ 趟冒泡, 并且元素交换的次数最多。初始序列为正序时, 进行 1 趟冒泡 (无交换) 就可结束算法。

02. C

冒泡排序始终在调整“逆序”, 因此交换次数为排列中逆序的个数。对逆序序列进行冒泡排序, 每个元素向后调整时都需要进行比较, 因此共需要比较 $5+4+3+2+1=15$ 次。

03. D

选择排序在每趟结束后可以确定一个元素的最终位置, 不对。插入排序, 第 i 趟后前 $i+1$ 个元素应该是有序的, 不对。第二趟 $\{20, 15\}$ 和 $\{21, 25\}$ 是反序的, 因此不是归并排序。快速排序每趟都将基准元素放在其最终位置, 然后以它为基准将序列划分为两个子序列。观察题中的排序过程, 可知是快速排序。

04. C

以 46 为基准元素, 首先从后往前扫描比 46 小的元素, 并与之进行交换, 而后从前往后扫描比 46 大的元素并将 46 与该元素交换, 得到 $(40, 46, 56, 38, 79, 84)$ 。此后, 继续重复从后往前扫描与从前往后扫描的操作, 直到 46 处于最终位置。

05. D

当待排序数据为基本有序时, 每次选取第 n 个元素为基准, 会导致划分区间分配不均匀, 不利于发挥快速排序算法的优势。相反, 当待排序数据分布较为随机时, 基准元素能将序列划分为两个长度大致相等的序列, 这时才能发挥快速排序的优势。

06. D

这里问的是平均性能, 选项 A、B 的平均性能都会达到 $O(n^2)$, 而希尔排序虽然大大降低了直接插入排序的时间复杂度, 但其平均性能不如快速排序。另外, 虽然众多排序算法的平均时间复杂度也是 $O(n \log_2 n)$, 但快速排序算法的常数因子是最小的。

07. A

若为插入排序, 则前三个元素应该是有序的, 显然不对。而冒泡排序和选择排序经过两趟排序后应该有两个元素处于最终位置 (最左/右端), 无论是按从小到大还是从大到小排序, 数据序列中都没有两个满足这样的条件的元素, 因此只可能选择选项 A。

【另解】先写出排好序的序列, 并和题中的序列进行对比。

题中序列: 2 1 4 9 8 10 6 20

已排好序序列: 1 2 4 6 8 9 10 20

在已排好序的序列中, 与题中序列相同的元素有 4、8 和 20, 最左和最右两个元素与题中的序列不同, 所以不可能是冒泡排序、选择排序或插入排序。

08. C

从后往前冒泡的过程为, 第一趟{1, 8, 9, 10, 4, 5, 6, 20, 2}, 第二趟{1, 2, 8, 9, 10, 4, 5, 6, 20}, 第三趟{1, 2, 4, 8, 9, 10, 5, 6, 20}, 第四趟{1, 2, 4, 5, 8, 9, 10, 6, 20}, 第五趟{1, 2, 4, 5, 6, 8, 9, 10, 20}, 经过第五趟冒泡后, 序列已经全局有序, 因此选择选项 C。实际每趟冒泡发生交换后可以判断是否会产生新的逆序对, 若不会产生, 则本趟冒泡之后序列全局有序, 所以最少 5 趟即可。

09. B

第一趟从左往右的排序结果为 4, 7, 3, 5, 6, 8, 9, 1, 2, 10; 第二趟从右往左的排序结果为 1, 4, 7, 3, 5, 6, 8, 9, 2, 10; 第三趟从左往右的排序结果为 1, 4, 3, 5, 6, 7, 8, 2, 9, 10; 第四趟从右往左的排序结果为 1, 2, 4, 3, 5, 6, 7, 8, 9, 10; 第五趟从左往右的排序结果为 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 此时序列已有序, 但仍需进行一趟无交换的排序才能确定序列已有序, 因此共需 6 趟排序。

10. A、D

当每趟的枢轴值都把表等分为长度相近的两个子表时, 速度是最快的; 当表本身已经有序或逆序时, 速度最慢。选项 D 中的序列已按关键字排好序, 因此它是最慢的, 而选项 A 中第一趟枢轴值 21 将表划分为两个子表{9, 17, 5}和{25, 23, 30}, 而后对两个子表划分时, 枢轴值再次将它们等分, 所以该序列是快速排序最优的情况, 速度最快。针对其他选项, 可以进行类似的分析。

11. B

对各序列分别执行一趟快速排序, 可做如下分析(以选项 A 为例): 枢轴值为 92, 因此 35 移动到第一个位置, 96 移动到第六个位置, 30 移动到第二个位置, 再将枢轴值移动到 30 所在的单元, 即第五个位置, 所以选项 A 中序列移动的次数为 4。同样, 可以分析出选项 B 中序列的移动次数为 8, 选项 C 中序列的移动次数为 4, 选项 D 中序列的移动次数为 2。

12. C

显然, 若按从小到大排序, 则最终有序的序列是{11, 18, 23, 68, 69, 73, 93}; 若按从大到小排序, 则最终有序的序列是{93, 73, 69, 68, 23, 18, 11}。对比可知说法 I、II 中没有处于最终位置的元素, 所以说法 I、II 都不可能。说法 III 中 73 和 93 处于从大到小排序后的最终位置, 而且 73 将序列分割成大于 73 和小于 73 的两部分, 所以说法 III 是有可能的。说法 IV 中 73 和 93 处于从小到大排列后的最终位置, 73 也将序列分割成大于 73 和小于 73 的两部分。

13. B、C

快速排序过程构成一个递归树, 递归深度即递归树的高度。枢轴值每次都子表等分时, 递归树的高为 $\log_2 n$; 枢轴值每次都是子表的最大值或最小值时, 递归树退化为单链表, 树高为 n 。

14. D

快速排序的最好情况是每次划分将待排序列划分为等长的两部分。因此, 第一趟将第 1 个元素与后面的 7 个元素进行比较, 将原序列划分为长度为 3 和 4 的两个子表, 比较 7 次; 第二趟对两个子表进行划分, 将长度为 3 的子表划分为长度为 1 的两个子表(不用继续划分), 比较 2 次, 将长度为 4 的子表划分为长度为 1 和 2 的两个子表, 比较 3 次; 第三趟将长度为 2 的子表划分为长度为 1 的子表, 比较 1 次。至此, 排序结束, 共进行的比较次数是 $7+2+3+1=13$ 。

15. D

考点追踪 ▶ 堆的删除操作及调整分析 (2015、2024)

每次输出堆顶后, 将堆底元素移至堆顶, 此时堆的性质可能被破坏, 需要自上而下地进行筛选: 将新堆顶与其左右孩子中的较大者比较, 若小于该子结点, 则交换; 重复此过程, 直至恢复堆性质。例如, 当 09 移至堆顶后, 先与左右孩子中的较大者 78 交换, 交换后破坏了 $L(3)$ 子树的堆; 继续对 $L(3)$ 子树向下筛选, 将 09 与 65 交换, 最终得到新堆, 调整过程如图 8.7 所示。

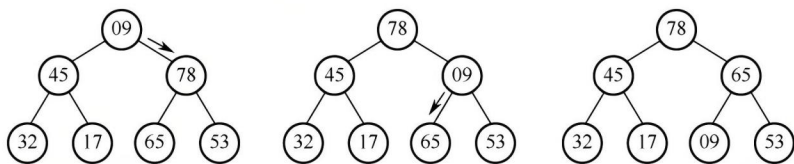


图 8.7 输出堆顶后再将剩余元素调整成新堆

建立大根堆的算法如下:

```
void BuildMaxHeap(ElemType A[], int len) {
    for (int i = len / 2; i > 0; i--) // 从最后一个分支结点到根结点
        HeapAdjust(A, i, len); // 反复调整子树
}

void HeapAdjust(ElemType A[], int k, int len) {
    // 对以 k 为根的子树进行调整
    A[0] = A[k]; // A[0] 暂存子树的根结点
    for (int i = 2 * k; i <= len; i *= 2) { // 沿关键字较大的子结点向下筛选
        if (i < len && A[i] < A[i + 1])
            i++; // 取关键字较大的子结点的下标
        if (A[0] >= A[i]) break; // 筛选结束
        else {
            A[k] = A[i]; // 将较大结点上移
            k = i; // 修改 k 值, 以便继续向下筛选
        }
    }
    A[k] = A[0]; // 被筛选结点的值放入最终位置
}
```

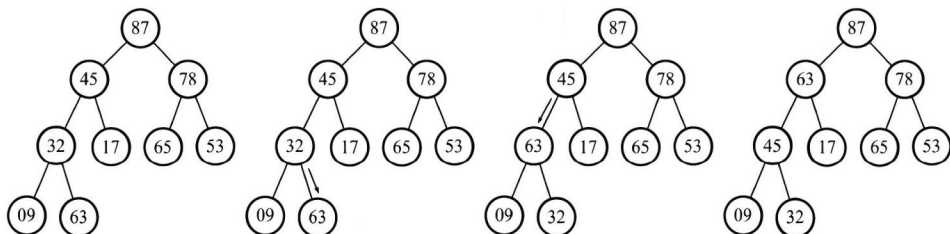
堆调整的时间复杂度与树高成正比, 为 $O(h)$ 。构建含 n 个元素的堆时, 关键字的总比较次数不超过 $4n$, 故建堆时间复杂度为 $O(n)$, 即可以在线性时间内将无序数组建成堆。

堆排序的完整算法如下:

```
void HeapSort(ElemType A[], int len) {
    BuildMaxHeap(A, len); // 初始建堆
    for (int i = len; i > 1; i--) { // 执行 n-1 趟排序
        Swap(A[i], A[1]); // 输出堆顶元素 (与堆底元素交换)
        HeapAdjust(A, 1, i - 1); // 把剩余 i-1 个元素重新整理为堆
    }
}
```

考点追踪 ▶ 堆的插入操作及调整操作分析 (2009、2011)

同时, 堆也支持插入操作。插入时, 先将新元素置于堆末端, 然后自下而上地与其父结点比较: 若违反堆性质, 则交换, 直至满足堆定义。图 8.8 展示了大根堆的插入操作示例。



(a)初始, 尾部插入 63 (b)63>32, 交换, 32 下沉 (c)63>45, 交换, 45 下沉

(d)调整完成

图 8.8 大根堆的插入操作示例

考点追踪 ▶ 堆在海量数据中选出最小 k 个数的应用及效率分析 (2022)

堆排序适用于大规模数据的排序场景, 还常用于解决 Top-K 问题。例如, 在 1 亿个数中选出最大的 100 个数。首先使用一个大小为 100 的数组, 读入前 100 个数, 构建小顶堆, 而后依次读入后续数字, 若小于堆顶则舍弃, 否则用该数替换堆顶并向下调整, 待数据读取完毕, 堆中 100 个数为所求。此方法的时间复杂度为 $O(n\log_2 k)$, 空间复杂度为 $O(k)$, 效率远高于全排序。

堆排序算法的性能分析如下。

空间效率: 仅使用了常数个辅助单元, 空间复杂度为 $O(1)$ 。

时间效率: 建堆的时间复杂度为 $O(n)$, 随后需进行 $n-1$ 次堆顶输出与调整操作, 每次调整的时间复杂度为 $O(\log_2 n)$ 。因此, 堆排序在最好、最坏和平均情况下的时间复杂度均为 $O(n\log_2 n)$ 。

稳定性: 在堆调整过程中, 相等关键字的元素可能因交换而改变其原始相对位置, 因此堆排序是不稳定的排序算法。例如, 表 $L = \{1, 2, 2\}$, 建堆时可能将 2 交换到堆顶, 此时 $L = \{2, 1, 2\}$, 最终排序序列为 $L = \{1, 2, 2\}$, 显然, 2 与 2 的相对次序已发生变化。

适用性: 堆排序依赖完全二叉树的随机访问特性, 因此仅适用于顺序存储的线性表。

8.4.3 本节试题精选

一、单项选择题

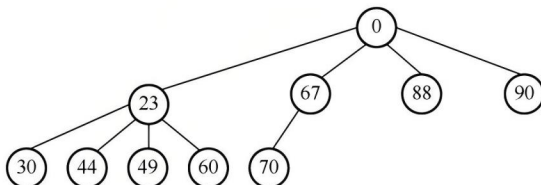
- 在以下排序算法中, 每趟从待排序记录中选出关键字最小的记录, 并将其与当前待排序部分的首记录交换, 从而使有序区从前向后逐步扩大的是 ()。
A. 简单选择排序 B. 冒泡排序 C. 堆排序 D. 直接插入排序
- 简单选择排序算法的比较次数和移动次数分别为 ()。
A. $O(n)$, $O(\log_2 n)$ B. $O(\log_2 n)$, $O(n^2)$
C. $O(n^2)$, $O(n)$ D. $O(n\log_2 n)$, $O(n)$
- 若只想得到 100000 个元素组成的序列中第 10 个最小元素之前的部分排序的序列, 用 () 方法最快。
A. 冒泡排序 B. 快速排序 C. 归并排序 D. 堆排序
- 下列 () 是一个堆。
A. 19, 75, 34, 26, 97, 56 B. 97, 26, 34, 75, 19, 56
C. 19, 56, 26, 97, 34, 75 D. 19, 34, 26, 97, 56, 75
- 在含有 n 个元素的小根堆中 (下标从 1 开始), 关键字最大的元素可能存储在 () 位置。
A. $n/2$ B. $n/2 + 2$ C. 1 D. $n/2 - 1$
- 向具有 n 个结点的堆中插入一个新元素的时间复杂度为 (), 删除一个元素的时间复杂度为 ()。
A. $O(1)$ B. $O(n)$ C. $O(\log_2 n)$ D. $O(n\log_2 n)$

07. 构建 n 个记录的初始堆, 其时间复杂度为 (); 对 n 个记录进行堆排序, 最坏情况下其时间复杂度为 ()。
- A. $O(n)$ B. $O(n^2)$ C. $O(\log_2 n)$ D. $O(n \log_2 n)$
08. 下列 4 种排序算法中, 排序过程中的比较次数与序列初始状态无关的是 ()。
- A. 简单选择排序 B. 直接插入排序 C. 快速排序 D. 冒泡排序
09. 对由相同的 n 个整数构成的二叉排序树和小根堆, 下列说法中不正确的是 ()。
- A. 二叉排序树的高度大于或等于小根堆的高度
B. 对二叉排序树进行中序遍历可以得到从小到大的序列
C. 从小根堆的根结点到任意叶结点的路径构成从小到大的序列
D. 对小根堆进行层序遍历可以得到从小到大的序列
10. 有一组数据(15, 9, 7, 8, 20, -1, 7, 4), 用堆排序的筛选方法建立的初始小根堆为 ()。
- A. -1, 4, 8, 9, 20, 7, 15, 7 B. -1, 7, 15, 7, 4, 8, 20, 9
C. -1, 4, 7, 8, 20, 15, 7, 9 D. A、B、C 均不对
11. 对关键字序列{23, 17, 72, 60, 25, 8, 68, 71, 52}进行堆排序, 输出两个最小关键字后的剩余堆为 ()。
- A. {23, 72, 60, 25, 68, 71, 52} B. {23, 25, 52, 60, 71, 72, 68}
C. {71, 25, 23, 52, 60, 72, 68} D. {23, 25, 68, 52, 60, 72, 71}
12. 堆排序分为两个阶段: 第一阶段将给定的序列构造成一个初始堆, 第二阶段逐次输出堆顶元素, 并调整使其保持堆的性质。设有给定序列{48, 62, 35, 77, 55, 14, 35, 98}, 若在堆排序的第一阶段将该序列构造成一个大根堆, 则交换元素的次数为 ()。
- A. 5 B. 6 C. 7 D. 8
13. 已知大根堆{62, 34, 53, 12, 8, 46, 22}, 删除堆顶元素后需要重新调整堆, 则在此过程中关键字的比较次数为 ()。
- A. 2 B. 3 C. 4 D. 5
14. 从根结点到任意叶结点的路径都是有序的数据结构是 ()。
- A. 红黑树 B. 二叉查找树 C. 哈夫曼树 D. 堆
15. 【2009 统考真题】已知关键字序列{5, 8, 12, 19, 28, 20, 15, 22}是小根堆, 插入关键字 3, 调整后得到的小根堆为 ()。
- A. 3, 5, 12, 8, 28, 20, 15, 22, 19 B. 3, 5, 12, 19, 20, 15, 22, 8, 28
C. 3, 8, 12, 5, 20, 15, 22, 28, 19 D. 3, 12, 5, 8, 28, 20, 15, 22, 19
16. 【2011 统考真题】已知序列{25, 13, 10, 12, 9}是大根堆, 在序列尾部插入新元素 18, 再将其调整为大根堆, 调整过程中元素之间进行的比较次数是 ()。
- A. 1 B. 2 C. 4 D. 5
17. 【2015 统考真题】已知小根堆为 8, 15, 10, 21, 34, 16, 12, 删除关键字 8 之后需重建堆, 在此过程中, 关键字之间的比较次数是 ()。
- A. 1 B. 2 C. 3 D. 4
18. 【2018 统考真题】在将序列(6, 1, 5, 9, 8, 4, 7)建成大根堆时, 正确的序列变化过程是 ()。
- A. 6, 1, 7, 9, 8, 4, 5 $\rightarrow$ 6, 9, 7, 1, 8, 4, 5 $\rightarrow$ 9, 6, 7, 1, 8, 4, 5 $\rightarrow$ 9, 8, 7, 1, 6, 4, 5
B. 6, 9, 5, 1, 8, 4, 7 $\rightarrow$ 6, 9, 7, 1, 8, 4, 5 $\rightarrow$ 9, 6, 7, 1, 8, 4, 5 $\rightarrow$ 9, 8, 7, 1, 6, 4, 5
C. 6, 9, 5, 1, 8, 4, 7 $\rightarrow$ 9, 6, 5, 1, 8, 4, 7 $\rightarrow$ 9, 6, 7, 1, 8, 4, 5 $\rightarrow$ 9, 8, 7, 1, 6, 4, 5
D. 6, 1, 7, 9, 8, 4, 5 $\rightarrow$ 7, 1, 6, 9, 8, 4, 5 $\rightarrow$ 7, 9, 6, 1, 8, 4, 5 $\rightarrow$ 9, 7, 6, 1, 8, 4, 5 $\rightarrow$ 9, 8, 6, 1, 7, 4, 5

19. 【2020 统考真题】下列关于大根堆（至少含 2 个元素）的叙述中，正确的是（ ）。
 I. 可以将堆视为一棵完全二叉树
 II. 可以采用顺序存储方式保存堆
 III. 可以将堆视为一棵二叉排序树
 IV. 堆中的次大值一定在根的下一层
 A. 仅 I、II B. 仅 II、III C. 仅 I、II 和 IV D. I、III 和 IV
20. 【2021 统考真题】将关键字 6,9,1,5,8,4,7 依次插入初始为空的大根堆 H, 得到的 H 是（ ）。
 A. 9,8,7,6,5,4,1 B. 9,8,7,5,6,1,4 C. 9,8,7,5,6,4,1 D. 9,6,7,5,8,4,1
21. 【2024 统考真题】已知关键字序列 28, 22, 20, 19, 8, 12, 15, 5 是大根堆（最大堆），对该堆进行两次删除操作后，得到的新堆为（ ）。
 A. 20, 19, 15, 12, 8, 5 B. 20, 19, 15, 5, 8, 12
 C. 20, 19, 12, 15, 8, 5 D. 20, 19, 8, 12, 15, 5

二、综合应用题

01. 指出堆和二叉排序树的区别？
02. 画出一棵二叉树，使得它既满足大根堆的要求又满足二叉排序树的要求。
03. 若只想得到一个序列中第 k ($k \geq 5$) 个最小元素之前的部分的排序序列，则最好采用什么排序算法？
04. 通常使用的堆也称二叉堆，因为它用完全二叉树来实现的，树中结点最多只有两个孩子。同理可以有 m 叉堆，即用完全 m 叉树来实现的堆。
 1) 下图是一个 m 叉小根堆，问 m 值是多少？向这个堆插入一个元素 65 后，堆中的元素如何变化？再删除堆顶元素呢？请画出变化后的树形。



- 2) 从 0 开始对完全 4 叉树中的结点从左到右、从上到下进行编号。若给定一个结点 k ，其父结点的编号是多少？（若存在），其第 i ($i=1, 2, 3, 4$) 个孩子的编号是多少？
- 3) 在 m 叉堆中进行插入和删除操作的时间复杂度是多少？
05. 编写一个算法，在基于单链表表示的待排序关键字序列上进行简单选择排序。
06. 试设计一个算法，判断一个数据序列是否构成一个小根堆。
07. 优先队列（Priority Queue）是一种数据结构，它类似于普通队列，但每个元素都有一个优先级。元素在入队时会根据其优先级来排序，而不按照先入先出的顺序来排序。每次从优先队列中出队时，出队的是优先级最高的元素，而不是最早进入队列的元素。

队列中的元素的数据结构的定义如下：

```

typedef struct{
    int value;           //元素的值
    int priority;        //元素的优先级, priority 越大, 优先级越高
}PriorityQueueElement;
  
```

请设计一个优先队列，要求满足：① 初始时队列为空；② 入队时，不允许增加队列的占用空间；③ 出队后，出队元素所占用的空间可重复使用，即整个队列所占用的空间不变；④ 入队操作和出队操作的时间复杂度始终保持为 $O(\log_2 n)$ 。请回答：

1) 该队列是应选择链式存储结构, 还是选择顺序存储结构?

2) 给出优先队列的数据结构的定义。

3) 用伪代码给出入队操作和出队操作的基本过程(关键之处可用文字描述)。

08. 【2022 统考真题】现有 n ($n > 100000$) 个数保存在一维数组 M 中, 需要查找 M 中最小的 10 个数。请回答下列问题。

1) 设计一个完成上述查找任务的算法, 要求平均情况下的比较次数尽可能少, 简述其算法思想(不需要编程实现)。

2) 说明你所设计的算法平均情况下的时间复杂度和空间复杂度。

8.4.4 答案与解析

一、单项选择题

01. A

该算法描述的是简单选择排序: 每趟从待排序部分顺序扫描, 选出最小关键字记录, 与首元素交换, 使有序区从前向后扩展。这一过程强调“选择最小”和“一次定位”。

02. C

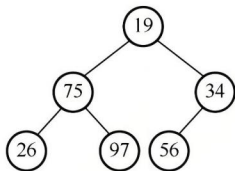
注意, 读者应熟练掌握各种排序算法的思想、过程和特点。

03. D

采用堆排序时, 读入前 10 个元素, 建立含 10 个元素的大根堆, 而后依次扫描剩余元素, 若大于堆顶, 则舍弃, 否则用该元素取代堆顶并重新调整堆, 当元素全部扫描完毕, 堆中保存的即是最小的 10 个元素。冒泡排序需要从后往前执行 10 趟冒泡才能得到 10 个最小的元素。两者的时间复杂度都和数据规模 n 线性相关, 但显然堆排序的常数系数更小。而快速排序、归并排序的每一趟都不能保证得到当前序列的最小值, 也无法达到线性时间复杂度。

04. D

可将每个选项中的序列表示成完全二叉树, 再看父结点与子结点的关系是否全部满足堆的定义。例如, 选项 A 中序列对应的完全二叉树如右图所示。显然, 最小元素 19 在根结点, 因此可能是小根堆, 但 75 与 26 的关系却不满足小根堆的定义, 所以选项 A 中的序列不是一个堆。其他选项采用类似的过程分析。



05. B

在小根堆中, 关键字最大的元素一定存储在该堆所对应的完全二叉树的叶结点中(位于最底层或次底层)。又因为二叉树中的最后一个非叶结点存储在 $\lfloor n/2 \rfloor$ 中, 所以关键字最大的元素的存储范围为 $\lfloor n/2 \rfloor + 1 \sim n$ 。注意, 读者可类比在大根堆中求关键字最小的元素的存储范围。

06. C、C

在向有 n 个元素的堆中插入一个新元素时, 需要调用一个向上调整的算法, 比较次数最多等于树的高度减 1, 因为树的高度为 $\lfloor \log_2 n \rfloor + 1$, 所以堆的向上调整算法的比较次数最多等于 $\lfloor \log_2 n \rfloor$ 。此处需要注意, 调整堆和建初始堆的时间复杂度是不一样的, 读者可以仔细分析两个算法的具体执行过程。

07. A、D

建堆过程中, 向下调整的时间与树高 h 有关, 为 $O(h)$ 。每次向下调整时, 大部分结点的高度都较小。因此, 可以证明在元素个数为 n 的序列上建堆, 其时间复杂度为 $O(n)$ 。无论是在最好情况下还是在最坏情况下, 堆排序的时间复杂度均为 $O(n \log_2 n)$ 。

08. A

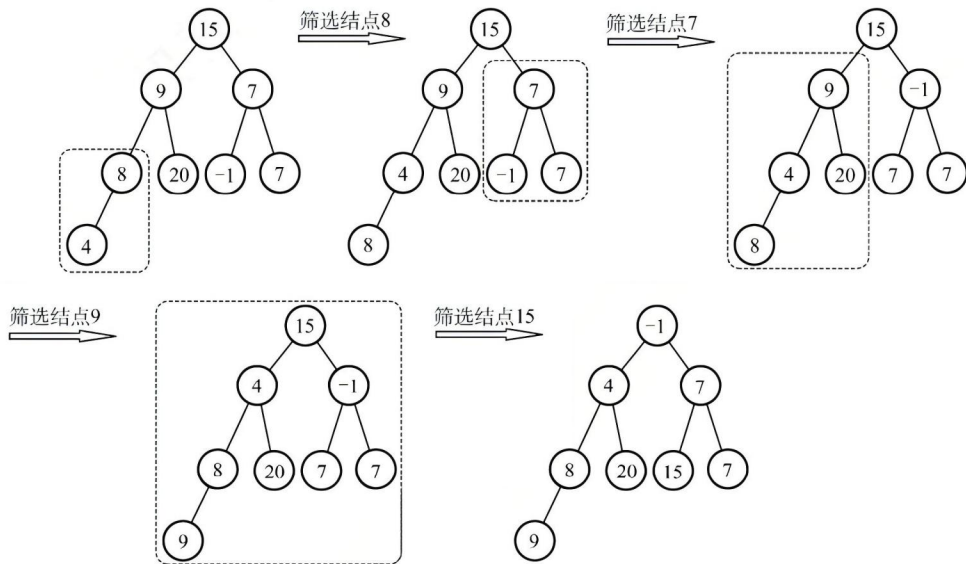
简单选择排序的比较次数始终为 $n(n-1)/2$ ，与序列状态无关。

09. D

堆是顺序存储的完全二叉树，因此其高度小于或等于结点数相同的二叉排序树，选项 A 正确。选项 B 显然正确。根据小根堆的定义，其根结点到任意叶结点的路径构成从小到大的序列，选项 C 正确。堆的各层结点之间没有大小要求，因此层序遍历不能保证得到有序序列，选项 D 错误。

10. C

从 $\lfloor n/2 \rfloor \sim 1$ 依次筛选堆的过程如下图所示，显然选择选项 C。



11. D

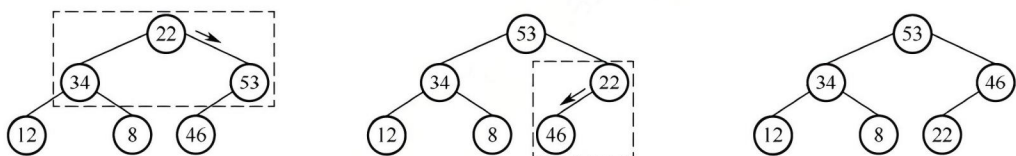
筛选法初始建堆为 $\{8, 17, 23, 52, 25, 72, 68, 71, 60\}$ ，输出 8 后重建的堆为 $\{17, 25, 23, 52, 60, 72, 68, 71\}$ ，输出 17 后重建的堆为 $\{23, 25, 68, 52, 60, 72, 71\}$ 。

12. B

初始序列是一棵顺序存储的完全二叉树，然后根据大根堆的要求，按照从下到上、从右到左的顺序进行调整。98 和 77 比较，98 和 77 交换（交换 1 次）；14 和 35 比较，35 和 35 比较，不交换；98 和 55 比较，98 和 62 比较，98 和 62 交换（交换 1 次）；62 和 77 比较，77 和 62 交换（交换 1 次）；98 和 35 比较，98 和 48 比较，98 和 48 交换（交换 1 次）；77 和 55 比较，77 和 48 比较，77 和 48 交换（交换 1 次）；48 和 62 比较，62 和 48 交换（交换 1 次），共交换 6 次。

13. B

删除堆顶 62 后，将堆尾 22 放入堆顶，然后自上而下调。首先 34 与 53 比较（第一次比较），较大者 53 与根 22 比较（第二次比较），53 被换至堆顶；22 只有一个孩子，直接与其左孩子 46 比较（第 3 次比较），22 与 46 交换，至此大根堆调整结束，具体过程如下图所示。



第三趟分配用最高位关键字 K^1 (百位) 进行, 将所有百位相等的记录分配到同一个队列, 如图 8.12(a) 所示, 第三趟收集后的结果如图 8.12(b) 所示, 至此整个排序结束。

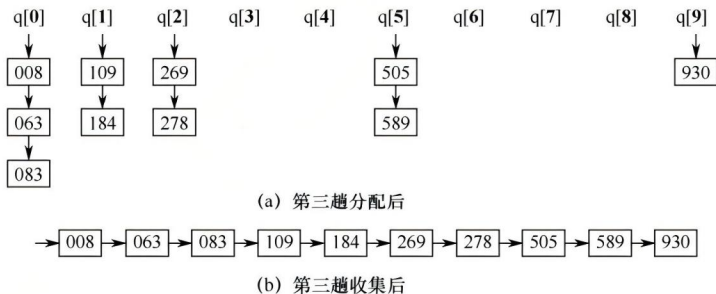


图 8.12 第三趟链式基数排序操作

基数排序算法的性能分析如下。

空间效率: 一趟排序需要的辅助空间为 r (r 个队列: r 个队头指针和 r 个队尾指针), 但这些结构在后续各趟排序中可重复使用, 因此空间复杂度为 $O(r)$ 。

考点追踪 ▶ 元素的移动次数与序列初态无关的排序算法 (2015)

时间效率: 共需进行 d 趟分配与收集。每趟分配需遍历 n 个元素, 所需时间为 $O(n)$; 每趟收集需处理 r 个队列, 所需时间为 $O(r)$ 。总时间复杂度为 $O(d(n+r))$, 它与序列的初始状态无关。

稳定性: 由于每趟分配按原序列顺序入队, 收集按队列编号顺序出队, 相同关键字的元素相对次序不会改变, 因此基数排序是稳定的排序算法。

适用性: 基数排序适用于顺序存储和链式存储的线性表, 链式结构尤其适合。

*8.5.3 计数排序

考点追踪 ▶ (算法题) 计数排序思想的应用 (2013、2015、2018)

计数排序也是一种不依赖于比较的排序算法。其核心思想是: 对每个待排序元素 x , 统计小于 x 的元素个数, 从而确定 x 在有序序列中的位置。例如, 若有 8 个元素小于 x , 则 x 应排在第 9 位。当存在重复元素时, 需对算法稍做调整, 以保证排序的稳定性。

注 意

计数排序并不在统考大纲的范围内, 但其排序思想在历年真题中多次涉及。

在计数排序的实现中, 假设输入是一个数组 $A[n]$, 序列长度为 n 。算法还需两个辅助数组: $B[n]$ 用于存放排序结果, $C[k]$ 用于存储计数值。具体做法是: 以 A 中的元素值作为 C 的下标 (索引), 而 $C[x]$ 保存的是值为 x 的元素出现的次数。算法实现如下:

考点追踪 ▶ 计数排序相关的思想和代码的分析 (2021)

```
void CountSort(ElemType A[], ElemType B[], int n, int k) {
    int i, C[k];
    for (i = 0; i < k; i++)
        C[i] = 0; // 初始化计数数组
    for (i = 0; i < n; i++)
        C[A[i]] = C[A[i]] + 1; // 遍历输入数组, 统计每个元素出现的次数
    for (i = 1; i < k; i++)
        C[i] = C[i] + C[i - 1]; // C[x] 保存的是小于或等于 x 的元素总数
    for (i = n - 1; i >= 0; i--) {
        B[C[A[i]] - 1] = A[i]; // 从后往前遍历输入数组
        C[A[i]]--; // 将元素 A[i] 放置到输出数组 B[] 的正确位置
    }
}
```

```

        C[A[i]] = C[A[i]] - 1;    //更新计数数组，确保相同元素的相对顺序
    }
}

```

第一个 for 循环将计数数组 C 初始化为 0。第二个 for 循环遍历输入数组 A ，对每个元素 $x=A[i]$ ，将 $C[x]$ 加 1；循环结束后， $C[x]$ 保存的是值等于 x 的元素个数。第三个 for 循环通过累加计算后，使 $C[x]$ 变为小于或等于 x 的元素总数，即 x 在最终有序序列中的最后可能出现的位置（从 1 开始计数）。第四个 for 循环从后往前遍历 A ：对于每个 $A[i]$ ，将其放入输出数组 B 的 $C[A[i]]-1$ 位置（下标从 0 开始），然后将 $C[A[i]]$ 减 1。若 A 中无重复元素，则 $C[A[i]]-1$ 即为 $A[i]$ 在 B 中的唯一正确位置；若存在重复元素，“从后往前遍历+计数递减”的策略确保在 A 中先出现的相同元素，在 B 中仍排在前面，从而保证排序的稳定性。

假设输入数组 $A[]=\{2, 4, 3, 0, 2, 3\}$ ，第二个 for 循环结束后，辅助数组 C 的状态如图 8.13(a)所示；第三个 for 循环结束后， C 的状态如图 8.13(b)所示。图 8.13(c)至图 8.13(h)分别展示了第四个 for 循环每次迭代后，输出数组 B 和辅助数组 C 的变化情况。

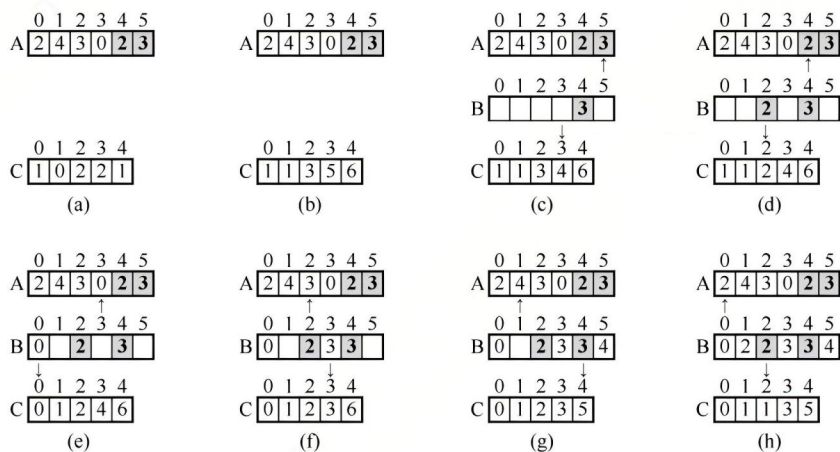


图 8.13 计数排序的过程

由上述过程可知，计数排序利用了数组下标天然有序的特性：将元素值映射为辅助数组的下标，通过统计频次并累加，再反向填充输出数组，从而完成排序。

计数排序算法的性能分析如下。

空间效率：计数排序是一种以空间换时间的算法。输出数组 B 的长度为 n ；计数数组 C 的长度为 k ，空间复杂度为 $O(n+k)$ 。若不将输出数组视为辅助空间，则空间复杂度为 $O(k)$ 。算法要求待排序元素为非负整数，取值范围（如 $0 \sim k-1$ ）不宜过大，否则会导致辅助空间浪费。

时间效率：上述代码的第 1 和第 3 个 for 循环耗时 $O(k)$ ，第 2 和第 4 个 for 循环耗时 $O(n)$ ，总时间复杂度为 $O(n+k)$ 。因此，当 $k=O(n)$ 时，计数排序可达线性时间 $O(n)$ ；但当 $k \gg O(n \log_2 n)$ 时，其效率反而不如基于比较的排序算法（如快速排序、堆排序等）。

稳定性：上述代码的第 4 个 for 循环从后往前遍历输入数组，相同元素在输出数组中的相对次序得以保持，因此计数排序是稳定的排序算法。

适用性：计数排序适用于顺序存储的线性表。

8.5.4 本节试题精选

一、单项选择题

01. 以下排序算法中，（ ）在一趟结束后不一定能选出一个元素放在其最终位置上。

- A. 简单选择排序 B. 冒泡排序 C. 归并排序 D. 堆排序
102. 以下排序算法中, () 不需要进行关键字的比较。
A. 快速排序 B. 归并排序 C. 基数排序 D. 堆排序
103. 在下列排序算法中, 平均情况下空间复杂度为 $O(n)$ 的是 (), 最坏情况下空间复杂度为 $O(n)$ 的是 ()。
I. 希尔排序 II. 堆排序 III. 冒泡排序
IV. 归并排序 V. 快速排序 VI. 基数排序
A. I、IV、VI B. II、V C. IV、V D. IV
104. 下列排序算法中, 排序过程中比较次数的数量级与序列初始状态无关的是 ()。
A. 归并排序 B. 插入排序 C. 快速排序 D. 冒泡排序
105. 2 路归并排序中, 归并趟数的数量级是 ()。
A. $O(n)$ B. $O(\log_2 n)$ C. $O(n \log_2 n)$ D. $O(n^2)$
106. 若对 27 个元素只进行三趟多路归并排序, 则选取的归并路数最少为 ()。
A. 2 B. 3 C. 4 D. 5
107. 将两个各有 N 个元素的有序表合并成一个有序表, 最少的比较次数是 (), 最多的比较次数是 ()。
A. N B. $2N-1$ C. $2N$ D. $N-1$
108. 用归并排序算法对序列 {1, 2, 6, 4, 5, 3, 8, 7} 进行排序, 共需要进行 () 次比较。
A. 12 B. 13 C. 14 D. 15
109. 一组经过第一趟 2 路归并排序后的记录的关键字为 {25, 50, 15, 35, 80, 85, 20, 40, 36, 70}, 其中包含 5 个长度为 2 的有序表, 用 2 路归并排序算法对该序列进行第二趟归并后的结果为 ()。
A. 15, 25, 35, 50, 80, 20, 85, 40, 70, 36 B. 15, 25, 35, 50, 20, 40, 80, 85, 36, 70
C. 15, 25, 50, 35, 80, 85, 20, 36, 40, 70 D. 15, 25, 35, 50, 80, 20, 36, 40, 70, 85
110. 若将中国人按照生日 (不考虑年份, 只考虑月、日) 来排序, 则使用下列排序算法时, 最快的是 ()。
A. 归并排序 B. 希尔排序 C. 快速排序 D. 基数排序
111. 设线性表中每个元素有两个数据项 k_1 和 k_2 , 现对线性表按以下规则进行排序: 先看数据项 k_1 , k_1 值小的元素在前, 大的元素在后; 在 k_1 值相同的情况下, 再看 k_2 , k_2 值小的元素在前, 大的元素在后。满足这种要求的排序算法是 ()。
A. 先按 k_1 进行直接插入排序, 再按 k_2 进行简单选择排序
B. 先按 k_2 进行直接插入排序, 再按 k_1 进行简单选择排序
C. 先按 k_1 进行简单选择排序, 再按 k_2 进行直接插入排序
D. 先按 k_2 进行简单选择排序, 再按 k_1 进行直接插入排序
112. 对 {05, 46, 13, 55, 94, 17, 42} 进行基数排序, 一趟排序的结果是 ()。
A. 05, 46, 13, 55, 94, 17, 42 B. 05, 13, 17, 42, 46, 55, 94
C. 42, 13, 94, 05, 55, 46, 17 D. 05, 13, 46, 55, 17, 42, 94
113. 有 n 个十进制整数进行基数排序, 其中最大的整数为 5 位, 则基数排序过程中临时建立的队列个数是 ()。
A. n B. 2 C. 5 D. 10
114. 下列各种排序算法中, () 需要的附加存储空间最大。

- A. 快速排序 B. 堆排序 C. 归并排序 D. 插入排序
15. 【2013 统考真题】已知两个长度分别为 m 和 n 的升序链表, 若将它们合并为长度为 $m+n$ 的一个降序链表, 则最坏情况下的时间复杂度为 ()。
- A. $O(n)$ B. $O(mn)$ C. $O(\min(m, n))$ D. $O(\max(m, n))$
16. 【2013 统考真题】对给定的关键字序列 110, 119, 007, 911, 114, 120, 122 进行基数排序, 第二趟分配、收集后得到的关键字序列为 ()。
- A. 007, 110, 119, 114, 911, 120, 122 B. 007, 110, 119, 114, 911, 122, 120
C. 007, 110, 911, 114, 119, 120, 122 D. 110, 120, 911, 122, 114, 007, 119
17. 【2015 统考真题】下列排序算法中, 元素的移动次数与关键字的初始状态无关的是 ()。
- A. 直接插入排序 B. 冒泡排序 C. 基数排序 D. 快速排序
18. 【2021 统考真题】设数组 $S[] = \{93, 946, 372, 9, 146, 151, 301, 485, 236, 327, 43, 892\}$, 采用最低位优先 (LSD) 基数排序将 S 排列成升序序列。第一趟分配、收集后, 元素 372 之前、之后紧邻的元素分别为 ()。
- A. 43, 892 B. 236, 301 C. 301, 892 D. 485, 301
19. 【2022 统考真题】使用 2 路归并排序对含 n 个元素的数组 M 进行排序时, 2 路归并排序的功能是 ()。
- A. 将两个有序表合并为一个新的有序表
B. 将 M 划分为两部分, 两部分的元素个数大致相等
C. 将 M 划分为 n 个部分, 每个部分中仅含有一个元素
D. 将 M 划分为两部分, 一部分元素的值均小于另一部分元素的值
20. 【2024 统考真题】现有由关键字组成的 3 个有序序列 (3, 5)、(7, 9) 和 (6), 若按从左至右的次序选择有序序列进行 2 路归并排序, 则关键字之间的总比较次数是 ()。
- A. 3 B. 4 C. 5 D. 6

二、综合应用题

01. 已知序列 {503, 87, 512, 61, 908, 170, 897, 275, 653, 462}, 采用非递归的 2 路归并排序算法对该序列做升序排序时需要几趟排序? 给出每一趟的结果。
02. 设待排序的关键字序列为 {12, 2, 16, 30, 28, 10, 16*, 20, 6, 18}, 试写出使用最低位优先 (LSD) 基数排序算法每趟排序后的结果。
03. 【2021 统考真题】已知某排序算法如下:

```
void cmpCountSort(int a[], int b[], int n){
    int i, j, *count;
    count = (int *)malloc(sizeof(int)*n);
    //C++语言: count = new int[n];
    for(i=0; i<n; i++)    count[i]=0;
    for(i=0; i<n-1; i++)
        for(j=i+1; j<n; j++)
            if(a[i]<a[j])    count[j]++;
            else            count[i]++;
    for(i=0; i<n; i++)    b[count[i]] = a[i];
    free(count);        //C++语言: delete count;
}
```

请回答下列问题。

- 1) 若有 $\text{int } a[] = \{25, -10, 25, 10, 11, 19\}$, $b[6]$; , 则调用 $\text{cmpCountSort}(a, b, 6)$ 后数组 b 中的内容是什么?
- 2) 若 a 中含有 n 个元素, 则算法执行过程中, 元素之间的比较次数是多少?

3) 该算法是稳定的吗? 若是, 阐述理由; 否则, 修改为稳定排序算法。

8.5.5 答案与解析

一、单项选择题

01. C

我们知道插入排序不能保证在一趟排序结束后一定有元素放在最终位置上。事实上, 归并排序也不能保证。例如, 序列{6, 5, 7, 8, 2, 1, 4, 3}进行一趟 2 路归并排序(从小到大)后为{5, 6, 7, 8, 1, 2, 3, 4}, 显然它们都未被放在最终位置上。

02. C

基数排序是基于关键字各位的大小进行排序的, 而不是基于关键字的比较进行的。

03. D、C

归并排序在平均情况和最坏情况下的空间复杂度都是 $O(n)$, 快速排序只在最坏情况下才是 $O(n)$, 平均情况是 $O(\log_2 n)$ 。因此, 归并排序是本章所有排序算法中占用辅助空间最多的。

04. A

前面已讲过选择排序的比较次数与序列初始状态无关, 归并排序的比较次数的数量级也与序列的初始状态无关。读者应能从算法的原理方面来考虑为什么和初始状态无关。

05. B

N 个元素进行 k 路归并排序的趟数 m 满足 $k^m = N$, 即 $m = \lceil \log_k N \rceil$, 本题中为 $\lceil \log_2 n \rceil$ 。

06. B

N 个元素进行 k 路归并排序的趟数 m 满足 $k^m = N$, 这里要求的是 k , 代入可得 $k = 3$ 。

07. A、B

注意, 当一个表中的最小元素比另一个表中的最大元素还大时, 比较的次数是最少的, 仅比较 N 次; 而当两个表中的元素依次间隔地比较时, 即 $a_1 < b_1 < a_2 < b_2 < \dots < a_n < b_n$ 时, 比较的次数是最多的, 为 $2N-1$ 次。

建议读者对此举一反三: 若将本题中的两个有序表的长度分别设为 M 和 N , 则最多(或最少)的比较次数是多少? 时间复杂度又是多少?

08. C

第一趟归并后{1, 2}, {4, 6}, {3, 5}, {7, 8}, 共比较 4 次; 第二趟归并后{1, 2, 4, 6}, {3, 5, 7, 8}, 共比较 4 次; 第三趟归并后{1, 2, 3, 4, 5, 6, 7, 8}, 共比较 6 次。三趟归并共需进行 14 次比较。

09. B

这里采用 2 路归并排序算法, 且是第二趟排序, 因此每 4 个元素放在一起归并, 可将序列划分为{25, 50, 15, 35}, {80, 85, 20, 40}和{36, 70}, 分别对它们进行排序后有{15, 25, 35, 50}, {20, 40, 80, 85}和{36, 70}。

10. D

按照所有中国人的生日排序, 一方面 N 是非常大的, 另一方面关键字所含的排序码数为 2, 且一个排序码的基数为 12, 另一个排序码的基数为 31, 都是较小的常数值, 因此采用基数排序可以在 $O(N)$ 内完成排序过程。

11. D

本题思路来自基数排序的 LSD, 首先应确定 k_1 、 k_2 的排序顺序, 若先排 k_1 再排 k_2 , 则排序结果不符合题意, 排除选项 A 和 C。再考虑排序的稳定性, 当 k_2 排好序后, 再对 k_1 排序, 若对 k_1 排序采

用的方法是不稳定的, 则对于 k_1 相同而 k_2 不同的元素可能会改变相对次序, 从而不一定能满足题设要求。直接插入排序是稳定的, 而简单选择排序是不稳定的, 只能选择选项 D。

12. C

基数排序有 MSD 和 LSD 两种, 基数排序是稳定的。对于选项 A, 不符合 LSD 和 MSD; 对于选项 B, 符合 MSD, 但关键字 42、46 的相对位置发生了变化; 对于选项 D, 不符合 LSD 和 MSD。

13. D

基数排序中建立的队列个数等于进制数。

14. C

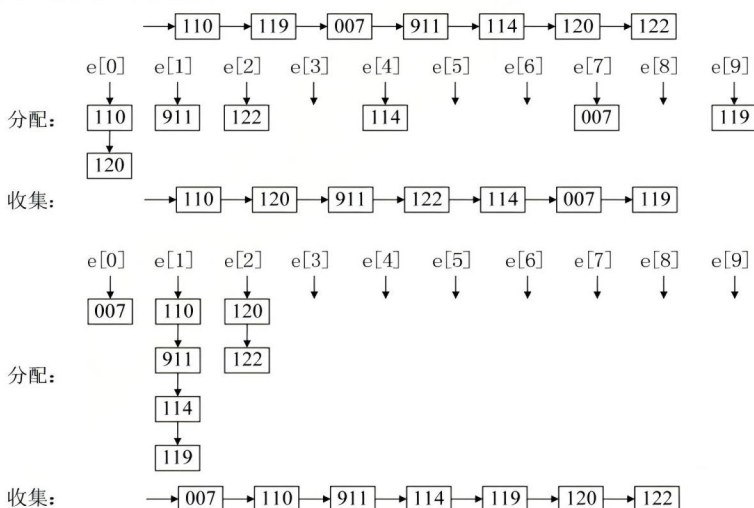
快速排序的平均空间复杂度是 $O(\log_2 n)$, 归并排序的空间复杂度是 $O(n)$, 其他都是 $O(1)$ 。

15. D

考虑两个升序链表合并, 两两比较表中元素, 每比较一次, 确定一个元素的链接位置 (取较小元素, 头插法)。当一个链表比较结束后, 将另一个链表的剩余元素插入即可。最坏的情况是两个链表中的元素依次进行比较, 因为 $2\max(m, n) \geq m + n$, 所以时间复杂度为 $O(\max(m, n))$ 。此外, 每次比较把两个链表中的较小结点插入新链表的表头, 直到一个链表为空, 因为原链表是升序排列的, 要求合并后为降序排列的, 因此还要把另一个链表剩下的结点一一插入新链表的表头, 不论是最好情况还是最坏情况, 都需要遍历两个链表中的所有结点。

16. C

基数排序的第一趟排序是按照个位数字的大小来进行的, 第二趟排序是按照十位数字的大小来进行的, 排序的过程如下图所示。



17. C

基数排序的元素移动次数与序列初态无关, 而其他三种排序算法都与序列初态有关。

18. C

基数排序是一种稳定的排序算法。因为采用最低位优先 (LSD) 的基数排序, 即第一趟对个位进行分配和收集操作, 所以第一趟分配和收集后的结果是 {151, 301, 372, 892, 93, 43, 485, 946, 146, 236, 327, 9}, 元素 372 之前、之后紧邻的元素分别为 301 和 892。

19. A。

送分题。本书对归并的定义原话是“归并的含义是将两个或两个以上的有序表合并成一个新的有序表”, 而 2 路归并是将两个有序表合并为一个新的有序表。

2) 由代码 `for(i=0;i<n-1;i++)` 和 `for(j=i+1;j<n;j++)` 可知, 在循环过程中, 每个元素都与它后面的所有元素比较一次(所有元素都两两比较一次), 比较次数之和为 $(n-1)+(n-2)+\cdots+1$, 所以总比较次数为 $n(n-1)/2$ 。

3) 不是。需要将程序中的 `if` 语句修改如下:

```
if(a[i]<=a[j]) count[j]++;  
else count[i]++;
```

若不加等号, 两个相等的元素比较时, 前面元素的 `count` 值会加 1, 则导致原序列中靠前的元素在排序后的序列中处于靠后的位置。

8.6 各种内部排序算法的比较及应用

8.6.1 内部排序算法的比较

前面讨论的排序算法很多, 各种排序算法的对比是考研常考的内容。通常从五个方面进行对比: 时间复杂度、空间复杂度、稳定性、适用性和过程特征。

考点追踪 ▶ 各种排序算法的特点和对比 (2009、2010、2017、2020、2022、2025)

从时间复杂度看: 简单选择排序、直接插入排序和冒泡排序在平均情况下的时间复杂度均为 $O(n^2)$, 且实现较为简单。其中, 直接插入排序和冒泡排序在最好情况下可达 $O(n)$, 而简单选择排序的时间复杂度与序列初始状态无关。希尔排序作为插入排序的改进, 对较大规模的数据通常具有较高效率, 但目前未知其精确的渐近时间。堆排序利用堆这种数据结构, 可在 $O(n)$ 时间内完成建堆, 整个排序过程的时间复杂度为 $O(n\log_2 n)$ 。快速排序基于分治思想, 最坏情况下的时间复杂度为 $O(n^2)$, 但平均性能可达 $O(n\log_2 n)$, 在实际应用中快速排序通常表现优异。归并排序同样基于分治思想, 其分割过程与初始序列无关, 因此最好、最坏和平均时间复杂度均为 $O(n\log_2 n)$ 。

从空间复杂度看: 简单选择排序、插入排序、冒泡排序、希尔排序和堆排序仅需常数级辅助空间, 空间复杂度为 $O(1)$ 。快速排序需要递归工作栈, 平均空间复杂度为 $O(\log_2 n)$, 最坏情况下可能达到 $O(n)$ 。2 路归并排序在合并操作中需要额外的辅助数组用于元素复制, 空间复杂度为 $O(n)$ 。虽有方法可减少辅助空间, 但通常会导致算法复杂度显著增加, 甚至影响时间效率。

考点追踪 ▶ 各种排序算法的稳定性及分析 (2021、2023)

从稳定性看: 插入排序、冒泡排序、归并排序和基数排序是稳定的排序算法, 而简单选择排序、快速排序、希尔排序和堆排序都是不稳定的。在平均时间复杂度为 $O(n\log_2 n)$ 的排序算法中, 只有归并排序是稳定的。判断排序算法是否稳定, 关键在于分析其操作是否会改变相等元素的相对次序; 对于不稳定的算法, 通常可通过构造一个反例加以验证。

考点追踪 ▶ 各种排序算法的适用性 (2017)

从适用性看: 折半插入排序、希尔排序、快速排序和堆排序适用于顺序存储。直接插入排序、冒泡排序、简单选择排序、归并排序和基数排序既适用于顺序存储, 也适用于链式存储。

考点追踪 ▶ 各种排序算法的过程特征 (2012)

从过程特征看: 不同排序算法在一趟或几趟处理后的中间结果通常是不同的。考研题目常给出初始序列和部分排序后的结果, 要求判断所用的排序方法。这就要求考生熟练掌握各类算法的过程特征。如冒泡排序、简单选择排序和堆排序在每趟处理后都能确定当前最大(或最小)元素的最终位置, 而快速排序在一趟划分后, 至少能确定一个元素(枢轴)的最终位置等。

表 8.1 列出了各种排序算法的时空复杂度和稳定性情况。其中,空间复杂度仅列出平均情况;由于希尔排序的时间复杂度依赖于所选的增量序列,因此无法给出精确表达式。

表 8.1 各种排序算法的性质

算法种类	时间复杂度			空间复杂度	是否稳定
	最好情况	平均情况	最坏情况		
直接插入排序	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	是
冒泡排序	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	是
简单选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	否
希尔排序				$O(1)$	否
快速排序	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n^2)$	$O(\log_2 n)$	否
堆排序	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(1)$	否
2 路归并排序	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n)$	是
基数排序	$O(d(n+r))$	$O(d(n+r))$	$O(d(n+r))$	$O(r)$	是

8.6.2 内部排序算法的应用

通常情况,对排序算法的比较和应用应考虑以下情况。

考点追踪 ▶ 选取排序算法时需要考虑的因素 (2019)

1. 选取排序算法需要考虑的因素

- 1) 待排序元素的个数 n 。
- 2) 待排序序列的初始状态 (如是否基本有序)。
- 3) 关键字的结构及其分布情况。
- 4) 对稳定性的要求。
- 5) 存储结构类型及辅助空间的限制等。

2. 排序算法选型小结

- 1) 若 n 较小,可采用直接插入排序或简单选择排序。直接插入排序的记录移动次数通常多于简单选择排序,因此当记录本身信息量较大时,简单选择排序更为合适。
- 2) 若 n 较大,应采用时间复杂度为 $O(n\log_2 n)$ 的排序算法,如快速排序、堆排序或归并排序。当关键字随机分布时,快速排序通常被认为是最高效的基于比较的内部排序算法。堆排序所需辅助空间少于快速排序,且不会退化到最坏情况,但两者均为不稳定排序。若既要求稳定性,又希望达到 $O(n\log_2 n)$ 的时间复杂度,则应选用归并排序。
- 3) 若初始序列已基本有序,则直接插入或冒泡排序更为高效,因其最好时间复杂度为 $O(n)$ 。
- 4) 在基于比较的排序算法中,每次比较两个关键字仅有两种可能的结果,因此其执行过程可用一棵判定树来描述。由此可证明:对于 n 个关键字的随机排列,任何基于比较的排序算法,在最坏情况下至少需要 $O(n\log_2 n)$ 的时间。
- 5) 若 n 很大,且关键字位数较少、可分解为多个子关键字 (如整数、字符串),则基数排序是更优选择,因其时间复杂度可接近线性。
- 6) 当记录本身信息量较大 (如包含大量字段或大对象) 时,为避免因频繁移动记录而产生高昂开销,宜采用链表作为存储结构,此时插入类或归并类排序更具优势。

8.6.3 本节试题精选

一、单项选择题

01. 若要求排序是稳定的,且关键字为实数,则在下列排序算法中应选 ()。

- A. 直接插入排序 B. 选择排序 C. 基数排序 D. 快速排序
02. 以下排序算法中时间复杂度为 $O(n\log_2 n)$ 且稳定的是 ()。
- A. 堆排序 B. 快速排序 C. 归并排序 D. 直接插入排序
03. 设被排序的结点序列共有 n 个结点, 在该序列中的结点已十分接近有序的情况下, 用直接插入排序、归并排序和快速排序对其进行排序, 这些算法的时间复杂度应为 ()。
- A. $O(n)$, $O(n)$, $O(n)$ B. $O(n)$, $O(n\log_2 n)$, $O(n\log_2 n)$
C. $O(n)$, $O(n\log_2 n)$, $O(n^2)$ D. $O(n^2)$, $O(n\log_2 n)$, $O(n^2)$
04. 下列排序算法中属于稳定排序的是 (①), 平均时间复杂度为 $O(n\log_2 n)$ 的是 (②), 在最好的情况下, 时间复杂度可以达到线性时间的有 (③)。(注: 多选题)
- I. 冒泡排序 II. 堆排序 III. 选择排序 IV. 直接插入排序
V. 希尔排序 VI. 归并排序 VII. 快速排序
05. 就排序算法所用的辅助空间而言, 堆排序、快速排序和归并排序的关系是 ()。
- A. 堆排序 < 快速排序 < 归并排序 B. 堆排序 < 归并排序 < 快速排序
C. 堆排序 > 归并排序 > 快速排序 D. 堆排序 > 快速排序 > 归并排序
06. 排序趟数与序列的初始状态无关的排序算法是 ()。
- I. 快速排序 II. 简单选择排序 III. 冒泡排序 IV. 基数排序
A. I、III B. II、IV C. I、II、IV D. I、IV
07. 排序趟数与序列的初始状态有关的排序算法是 ()。
- A. 直接插入排序 B. 2路归并排序 C. 快速排序 D. 堆排序
08. 对 n 个元素进行排序, 其排序趟数肯定为 $n-1$ 趟的排序算法是 ()。
- A. 直接插入排序和快速排序 B. 冒泡排序和快速排序
C. 简单选择排序和直接插入排序 D. 简单选择排序和冒泡排序
09. 若序列的初始状态为 {1, 2, 3, 4, 5, 10, 6, 7, 8, 9}, 要想使得排序过程中的元素比较次数最少, 则应该采用 () 方法。
- A. 插入排序 B. 选择排序 C. 希尔排序 D. 冒泡排序
10. 对于元素个数相同的不同初始序列, 总比较次数大致相同的排序算法是 ()。
- A. 折半插入排序和简单选择排序 B. 基数排序和归并排序
C. 冒泡排序和快速排序 D. 堆排序
11. 一般情况下, 以下查找效率最低的数据结构是 ()。
- A. 有序顺序表 B. 二叉排序树 C. 堆 D. 平衡二叉树
12. 一台计算机具有多核 CPU, 可以同时执行相互独立的任务。归并排序的各个归并段可以并行执行, 在下列排序算法中, 不可以并行执行的有 ()。
- I. 基数排序 II. 快速排序 III. 冒泡排序 IV. 堆排序
A. I、III B. I、II C. I、III、IV D. II、IV
13. 【2009 统考真题】若数据元素序列 {11, 12, 13, 7, 8, 9, 23, 4, 5} 是采用下列排序算法之一得到的第二趟排序后的结果, 则该排序算法只能是 ()。
- A. 冒泡排序 B. 插入排序 C. 选择排序 D. 2路归并排序
14. 【2010 统考真题】对一组数据 (2, 12, 16, 88, 5, 10) 进行排序, 若前 3 趟排序结果如下:
第一趟排序结果: 2, 12, 16, 5, 10, 88
第二趟排序结果: 2, 12, 5, 10, 16, 88
第三趟排序结果: 2, 5, 10, 12, 16, 88

则采用的排序算法可能是 ()。

A. 冒泡排序 B. 希尔排序 C. 归并排序 D. 基数排序

15. 【2012 统考真题】在内部排序过程中,对尚未确定最终位置的所有元素进行一遍处理称为一趟排序。下列排序算法中,每趟排序结束都至少能够确定一个元素最终位置的方法是 ()。

I. 简单选择排序 II. 希尔排序 III. 快速排序 IV. 堆排序 V. 2 路归并排序

A. 仅 I、III、IV B. 仅 I、III、V C. 仅 II、III、IV D. 仅 III、IV、V

16. 【2017 统考真题】在内部排序时,若选择了归并排序而未选择插入排序,则可能的理由是 ()。

I. 归并排序的程序代码更短 II. 归并排序的占用空间更少

III. 归并排序的运行效率更高

A. 仅 II B. 仅 III C. 仅 I、II D. 仅 I、III

17. 【2017 统考真题】下列排序算法中,若将顺序存储更换为链式存储,则算法的时间效率会降低的是 ()。

I. 插入排序 II. 选择排序 III. 冒泡排序 IV. 希尔排序 V. 堆排序

A. 仅 I、II B. 仅 II、III C. 仅 III、IV D. 仅 IV、V

18. 【2019 统考真题】选择一个排序算法时,除算法的时空效率外,下列因素中,还需要考虑的是 ()。

I. 数据的规模 II. 数据的存储方式 III. 算法的稳定性 IV. 数据的初始状态

A. 仅 III B. 仅 I、II C. 仅 II、III、IV D. I、II、III、IV

19. 【2020 统考真题】对大部分元素已有序的数组排序时,直接插入排序比简单选择排序效率更高,其原因是 ()。

I. 直接插入排序过程中元素之间的比较次数更少

II. 直接插入排序过程中所需的辅助空间更少

III. 直接插入排序过程中元素的移动次数更少

A. 仅 I B. 仅 III C. 仅 I、II D. I、II 和 III

20. 【2022 统考真题】对数据进行排序时,若采用直接插入排序而不采用快速排序,则可能的原因是 ()。

I. 大部分元素已有序 II. 待排序元素数量很少

III. 要求空间复杂度为 $O(1)$ IV. 要求排序算法是稳定的

A. 仅 I、II B. 仅 III、IV C. 仅 I、II、IV D. I、II、III、IV

21. 【2023 统考真题】下列排序算法中,不稳定的是 ()。

I. 希尔排序 II. 归并排序 III. 快速排序 IV. 堆排序 V. 基数排序

A. 仅 I、II B. 仅 II、V C. 仅 I、III、IV D. 仅 III、IV、V

22. 【2025 统考真题】在下列排序算法中,最坏情况下元素移动次数最少的是 ()。

A. 起泡排序 B. 直接插入排序 C. 快速排序 D. 简单选择排序

23. 【2025 统考真题】对含 9 个关键字的初始序列进行排序,若序列的变化情况如下表所示,则下列排序算法中,采用的是 ()。

初始序列	5, 25, 40, 30, 10, 20, 45, 15, 35
第 1 趟排序后的序列	5, 10, 20, 30, 15, 35, 45, 25, 40
第 2 趟排序后的序列	5, 10, 15, 25, 20, 30, 40, 35, 45

- A. 希尔排序 B. 基数排序 C. 归并排序 D. 折半插入排序

二、综合应用题

01. 设关键字序列为{3, 7, 6, 9, 7, 1, 4, 5, 20}, 对其进行排序的最小交换次数是多少?
02. 设顺序表用数组 $A[]$ 表示, 表中元素存储在数组下标 $1 \sim m+n$ 的范围内, 前 m 个元素递增有序, 后 n 个元素递增有序, 设计一个算法, 使得整个顺序表有序。
- 1) 给出算法的基本设计思想。
- 2) 根据设计思想, 采用 C/C++ 描述算法, 关键之处给出注释。
- 3) 说明你所设计算法的时间复杂度与空间复杂度。
03. 设有一个数组中存放了一个无序的关键序列 $K_1, K_2, \dots, K_n$ 。现要求将 K_n 放在将元素排序后的正确位置上, 试编写实现该功能的算法, 要求比较关键字的次数不超过 n 。

8.6.4 答案与解析

一、单项选择题

01. A

采用排除法。题目要求是稳定排序, 因此排除选项 B 和 D, 又基数排序不能对 float 型和 double 型的实数进行排序, 因此排除选项 C。

02. C

堆排序和快速排序不是稳定排序算法, 而直接插入排序算法的时间复杂度为 $O(n^2)$ 。

03. C

各种排序算法的时间和空间复杂度、稳定性等见表 8.1。

04. ① I、IV、VI ② II、VI、VII ③ I、IV

读者应能从算法的原理上理解算法的稳定性情况。堆排序和归并排序在最坏情况下的时间复杂度与最好情况下的时间复杂度是同一数量级的, 都是 $O(n \log_2 n)$ 。

05. A

堆排序的空间复杂度为 $O(1)$, 因此快速排序的空间复杂度在最坏情况下为 $O(n)$, 平均空间复杂度为 $O(\log_2 n)$, 归并排序的空间复杂度为 $O(n)$ 。

06. B

冒泡排序的趟数为 $1 \sim n-1$, 和序列初态有关。简单选择排序每趟都选出一个最小(或最大)的元素, 排序趟数固定为 $n-1$ 。基数排序每趟都要进行分配和收集, 排序趟数固定为 d 。快速排序的趟数和枢轴的选取有关, 即和划分是否对称有关, 当划分的两个区域分别包含 0 个元素和 $n-1$ 个元素时, 这种最大限度的不对称性发生在序列初态基本有序时。

07. C

直接插入排序的趟数始终为 $n-1$, 而与序列的初始状态无关。2 路归并排序的趟数始终为 $\lceil \log_2 n \rceil$, 而与序列的初始状态无关。堆排序每趟选出一个最大元素或最小元素, 然后调整堆, 初始建好堆后, 需要 $n-1$ 趟输出和调整堆的操作, 而与序列的初始状态无关。

08. C

简单选择排序和直接插入排序的趟数始终为 $n-1$ 。冒泡排序的趟数为 $1 \sim n-1$, 快速排序的趟数为 $\log_2 n \sim n-1$, 具体取决于序列的初始状态(快速排序还取决于划分方法)。

09. A

选择排序和序列初态无关, 直接排除。初始序列基本有序时, 插入排序比较次数较少。本题中, 插入排序仅需比较 $n-1+4$ 次, 而希尔排序和冒泡排序的比较次数均远大于此。

10. A

简单选择排序的总比较次数显然是确定的。折半插入排序每趟的比较次数都为 $O(\log_2 m)$ (m 为当前已排好序的子序列的长度), 因此总比较次数也是确定的。基数排序不是基于比较的排序算法。其他几种排序算法的比较次数显然和序列的初始状态有关。

11. C

堆是用于排序的, 在查找时它是无序的, 所以效率没有其他查找结构的高。

12. A

基数排序每趟需要利用前一趟已排好的序列, 无法并行执行。快速排序每趟划分的子序列互不影响, 可以并行执行。冒泡排序每趟对未排序的所有元素进行一趟处理, 无法并行执行。堆排序可以并行执行, 因为根结点的左右子树构成的子堆在执行过程中是互不影响的。

13. B

每趟冒泡和选择排序后, 总会有一个元素被放置在最终位置上。显然, 这里 {11, 12} 和 {4, 5} 所处的位置并不是最终位置, 因此不可能是冒泡和选择排序。2 路归并算法经过第二趟后应该是每 4 个元素有序的, 但 {11, 12, 13, 7} 并非有序, 因此也不可能是 2 路归并排序。

14. A

题中给出的排序过程, 每一趟都是从前往后依次比较使最大值沉底, 符合冒泡排序的特点。分别用其他 3 种排序算法尝试, 归并排序第一趟后的结果为 (2, 12, 16, 88, 5, 10), 基数排序第一趟后的结果为 (10, 2, 12, 5, 16, 88), 希尔排序显然不符合。

15. A

对于说法 I, 简单选择排序每次选择未排序序列中的最小元素放入其最终位置。对于说法 II, 希尔排序每次对划分的子表进行排序, 得到局部有序的结果, 所以不能保证每趟结束都能确定一个元素的最终位置。对于说法 III, 快速排序每趟结束后都将枢轴元素放到最终位置。对于说法 IV, 堆排序属于选择排序, 每次都把大根堆的根结点与表尾结点交换, 确定其最终位置。对于选项 V, 2 路归并排序每趟对子表进行两两归并, 从而得到若干局部有序的结果, 但无法确定最终位置。

16. B

归并排序的代码比插入排序的代码更为复杂, 前者的空间复杂度为 $O(n)$, 后者为 $O(1)$ 。但是前者的时间复杂度为 $O(n \log_2 n)$, 后者为 $O(n^2)$ 。

17. D

在顺序存储的条件下, 插入排序、选择排序、冒泡排序的时间复杂度都是 $O(n^2)$, 更换为链式存储后的时间复杂度还是 $O(n^2)$ 。希尔排序和堆排序都利用了顺序存储的随机访问特性, 而链式存储不支持这种性质, 所以时间复杂度会增加。

18. D

当数据规模较小时可选择复杂度为 $O(n^2)$ 的简单排序算法, 当数据规模较大时应选择复杂度为 $O(n \log_2 n)$ 的排序算法, 当数据规模大到内存无法放下时需选择外部排序算法, 说法 I 正确。数据的存储方式主要分为顺序存储和链式存储, 有些排序算法 (如堆排序) 只能用于顺序存储方式, 说法 II 正确。若对数据稳定性有要求, 则不能选择不稳定的排序算法, 说法 III 显然正确。当数据初始基本有序时, 直接插入排序的效率最高, 冒泡排序和直接插入排序的时间复杂度都为 $O(n)$, 而归并排序的时间复杂度仍然为 $O(n \log_2 n)$, 说法 IV 正确。

19. A

考虑比较极端的情况, 对于有序数组, 直接插入排序的比较次数为 $n-1$, 简单选择排序的比较次数始终为 $1+2+\cdots+n-1=n(n-1)/2$, 说法 I 正确。两种排序算法的辅助空间都为 $O(1)$, 无

8.7.5 最佳归并树

文件经过置换-选择排序后, 得到长度不等的初始归并段。下面讨论如何安排这些长度不等的归并段的归并顺序, 使得 I/O 次数最少。假设通过置换-选择排序得到 9 个初始归并段, 其长度依次为 9, 30, 12, 18, 3, 17, 2, 6, 24。若采用 3 路平衡归并, 一种可能的归并树如图 8.18 所示。

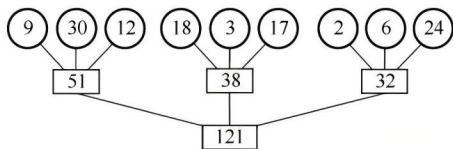


图 8.18 3 路平衡归并的归并树

在图 8.18 中, 各叶结点表示一个初始归并段, 其上的权值表示该段的长度; 叶结点到根的路径长度表示该段参与归并的趟数; 非叶结点表示归并生成的新段; 根结点对应最终有序文件。树的带权路径长度 WPL 等于归并过程中读取的总记录数, 因此总的 I/O 次数 = $2 \times WPL = 484$ 。

考点追踪 ▶ 构造三叉哈夫曼树及相关的分析和计算 (2013)

显然, 不同的归并方案对应不同的归并树, 其 WPL (I/O 次数) 也不同。为最小化 I/O 开销, 可将哈夫曼编码的思想推广至 k 叉树情形: 优先归并长度较小的段, 延后归并长度较大的段, 从而构造出 I/O 次数最少的最佳归并树。对上述 9 个归并段, 按此原则构造的最佳 3 路归并树如图 8.19 所示, 按此方案归并, 总 I/O 次数仅为 446, 显著优于普通方案。

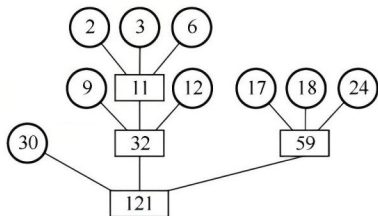


图 8.19 3 路平衡归并的最佳归并树

图 8.19 所示的归并树是一棵严格三叉树, 即树中只有度为 3 或 0 的结点。然而, 当初始归并段的数量不足以构成严格 k 叉树 (也称正则 k 叉树) 时, 直接进行 k 路归并会导致某些归并轮次不足 k 路, 从而降低效率。例如, 若只有 8 个初始归并段 (如删除上例中长度为 30 的段), 如果强行在最后一次归并时降为 2 路, 其余仍为 3 路, 则 I/O 次数将高达 386, 这并非最优方案。更优的做法是: 引入若干长度为 0 的“虚段” (不包含实际数据的空归并段), 使每趟归并都能满路 (每次归并都处理 3 个段)。这些虚段不会产生实际的 I/O 开销, 但能让归并树的结构更接近哈夫曼最优形态——权值较小的段 (包括虚段) 被安排在更深的位置, 避免过早参与归并。按照这一思路构造的最佳归并树如图 8.20 所示, 其 I/O 次数显著降低至 326。

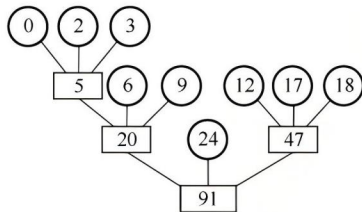


图 8.20 8 个归并段的最佳归并树

如何判定添加虚段的数目?

设度为 0 的结点有 n_0 个, 度为 k 的结点有 n_k 个, 归并树的结点总数为 n , 则有:

- $n = n_k + n_0$ (总结点数 = 度为 k 的结点数 + 度为 0 的结点数)
- $n = kn_k + 1$ (总结点数 = 所有结点的度数之和 + 1)

因此, 对严格 k 叉树有 $n_0 = (k-1)n_k + 1$, 由此可得 $n_k = (n_0 - 1)/(k-1)$ 。

接下来, 根据 n_0 和 k 的关系, 判断是否需要添加虚段:

- 若 $(n_0 - 1) \% (k - 1) = 0$ (%为取模运算), 则说明这 n_0 个叶结点 (初始归并段) 正好可以构成一棵严格的 k 叉归并树。此时, 内结点有 n_k 个。
- 若 $(n_0 - 1) \% (k - 1) = u \neq 0$, 则说明对于这 n_0 个叶结点, 其中有 u 个多余, 不能直接纳入 k 叉归并树。为构造包含所有 n_0 个初始归并段的 k 叉归并树, 应在原有 n_k 个内结点的基础上再增加 1 个内结点。该内结点将代替一个叶结点的位置, 被代替的叶结点连同多出的 u 个叶结点, 再加上 $k - u - 1$ 个空归并段, 即可构建完整的归并树。

考点追踪 ▶ 实现最佳归并时需补充的虚段数量的分析 (2019)

以图 8.20 为例, 用 8 个归并段构造二叉树时, $(n_0 - 1) \% (k - 1) = (8 - 1) \% (3 - 1) = 1 \neq 0$, 表明当前叶结点数无法构成严格的二叉树。因此, 需要添加 $3 - 1 - 1 = 1$ 个虚段, 使叶结点总数增至 9, 满足 $(9 - 1) \% (3 - 1) = 0$, 这样就可以构建一棵完整的严格二叉树。

8.7.6 本节试题精选

一、单项选择题

01. 外部排序和内部排序的主要区别是 ()。
 - A. 内部排序的数据量小, 而外部排序的数据量大
 - B. 内部排序不涉及内、外存数据交换, 而外部排序涉及内、外存数据交换
 - C. 内部排序的速度快, 而外部排序的速度慢
 - D. 内部排序所需的内存小, 而外部排序所需的内存大
02. 下列关于外部排序的说法中, 正确的是 ()。
 - A. 置换选择排序得到的初始归并段的长度一定相等
 - B. 内外存交换数据的时间只占总排序时间的一小部分
 - C. 败者树是一棵完全二叉树
 - D. 外部排序不涉及对文件的读/写操作
03. 多路平衡归并的作用是 ()。
 - A. 减少归并趟数
 - B. 减少初始归并段的个数
 - C. 便于实现败者树
 - D. 以上都对
04. 设在磁盘上存放有 375000 个记录, 进行 5 路平衡归并排序, 内存工作区能容纳 600 个记录, 为把所有记录排好序, 需要进行 () 趟归并排序。
 - A. 3
 - B. 4
 - C. 5
 - D. 6
05. 在下列关于外部排序过程输入/输出缓冲区作用的叙述中, 不正确的是 ()。
 - A. 暂存输入/输出记录
 - B. 内部归并的工作区
 - C. 产生初始归并段的工作区
 - D. 传送用户界面的消息
06. 若只需 3 趟排序就可完成 64 个元素的多路归并排序, 则选取的归并路数最少是 ()。
 - A. 2
 - B. 3
 - C. 4
 - D. 5
07. 置换-选择排序的作用是 ()。
 - A. 用于生成外部排序的初始归并段
 - B. 完成将一个磁盘文件排序成有序文件的有效的外部排序算法
 - C. 生成的初始归并段的长度是内存工作区的 2 倍
 - D. 对外部排序中输入/归并/输出的并行处理
08. 一个无序文件的 n 个记录采用置换选择排序产生 m 个有序段, 则 m 和 n 的关系是 ()。
 - A. m 与 n 成正比
 - B. $m = \log_2 n$
 - C. m 与 n 成反比
 - D. 以上都不对
09. 在由 k 路归并构建的败者树中选取一个关键字最小的记录, 则所需时间为 ()。
 - A. $O(1)$
 - B. $O(k)$
 - C. $O(\log_2 k)$
 - D. 以上都不对
10. 下列关于小顶堆和败者树的说法中, 正确的是 ()。
 - I. 败者树从下往上维护, 每上一层, 只需和失败结点比较 1 次
 - II. 败者树的每次维护, 必定要从叶结点一直走到根结点, 不可能从中间停止
 - III. 堆从上往下维护, 每下一层, 若其左右孩子均不为空, 则需比较 2 次

- IV. 堆的每次维护, 必定要从根结点一直走到叶结点, 不可能从中间停止
A. I、III B. II、III C. I、II、III D. I、III、IV
11. 最佳归并树在外部排序中的作用是 ()。
A. 完成 m 路归并排序 B. 设计 m 路归并排序的优化方案
C. 产生初始归并段 D. 与锦标赛树的作用类似
12. 在由 m 个初始归并段构建的 k 阶最佳归并树中, 不需要补充虚段, 则度为 k 的结点个数是 ()。
A. $(m-1)/k$ B. m/k C. $(m-1)/(k-1)$ D. 无法确定
13. 【2013 统考真题】已知二叉树 T 中 6 个叶结点的权分别是 2, 3, 4, 5, 6, 7, T 的带权 (外部) 路径长度最小是 ()。
A. 27 B. 46 C. 54 D. 56
14. 【2016 统考真题】对 10TB 的数据文件进行排序, 应使用的方法是 ()。
A. 希尔排序 B. 堆排序 C. 快速排序 D. 归并排序
15. 【2019 统考真题】设外存上有 120 个初始归并段, 进行 12 路归并时, 为实现最佳归并, 需要补充的虚段个数是 ()。
A. 1 B. 2 C. 3 D. 4
16. 【2024 统考真题】在外排序中, 利用败者树对初始为升序的归并段进行多路归并, 败者树中记录“冠军”的结点保存的是 ()。
A. 最大关键字 B. 最小关键字
C. 最大关键字所在的归并段号 D. 最小关键字所在的归并段号

二、综合应用题

01. 若某个文件经内部排序得到 80 个初始归并段, 试问:
1) 若使用多路平衡归并执行 3 趟完成排序, 则应取得的归并路数至少应为多少?
2) 若操作系统要求一个程序同时可用的输入/输出文件的总数不超过 15 个, 则按多路归并至少需要几趟可以完成排序? 若限定这个趟数, 可取的最低路数是多少?
02. 假设文件有 4500 个记录, 在磁盘上每个块可放 75 个记录。计算机中用于排序的内存区可容纳 450 个记录。试问:
1) 可以建立多少个初始归并段? 每个初始归并段有多少记录? 存放于多少个块中?
2) 应采用几路归并? 请写出归并过程及每趟需要读/写磁盘的块数。
03. 设初始归并段为 (10, 15, 31), (9, 20), (22, 34, 37), (6, 15, 42), (12, 37), (84, 95)。试利用败者树进行 m 路归并, 手工执行选择最小的 5 个关键字的过程。
04. 给出 12 个初始归并段, 其长度分别为 30, 44, 8, 6, 3, 20, 60, 18, 9, 62, 68, 85。现要做 4 路外归并排序, 试画出表示归并过程的最佳归并树, 并计算该归并树的带权路径长度 WPL。
05. 【2023 统考真题】对含有 n ($n > 0$) 个记录的文件进行外部排序, 采用置换-选择排序生成初始归并段时需要使用一个工作区, 工作区中能保存 m 个记录。请回答:
1) 若文件中含有 19 个记录, 其关键字依次是 51, 94, 37, 92, 14, 63, 15, 99, 48, 56, 23, 60, 31, 17, 43, 8, 90, 166, 100, 则当 $m=4$ 时, 可生成几个初始归并段? 各是什么?
2) 对任意的 m ($n \gg m > 0$), 生成的第一个初始归并段的长度最大值和最小值分别是多少?

8.7.7 答案与解析

一、单项选择题

01. B

外部排序和内部排序最主要的区别是是否涉及内存、外存的数据交换。

02. C

置换选择排序得到的是长度不一定相等的归并段, 选项 A 错误。外部排序的主要时间消耗在内外存之间的数据交换上, 选项 B 错误。败者树是一棵完全二叉树, 选项 C 正确。外部排序包括两个阶段: 生成初始归并段和对初始归并段进行归并, 这两个阶段都涉及对文件的读/写操作, 选项 D 错误。

03. A

多路平衡归并的目的是减少归并趟数, 因为当 m 个初始归并段采用 k 路平衡归并时, 所需趟数 $s = \lceil \log_k m \rceil$, 若不采用多路平衡归并, 则其归并趟数大于 s 。

04. B

初始归并段的个数 $r = 375000/600 = 625$, 因此, 归并趟数 $S = \lceil \log_m r \rceil = \lceil \log_5 625 \rceil = 4$ 。第一趟把 625 个归并段归并成 $625/5 = 125$ 个; 第二趟把 125 个归并段归并成 $125/5 = 25$ 个; 第三趟把 25 个归并段归并成 $25/5 = 5$ 个; 第四趟把 5 个归并段归并成 $5/5 = 1$ 个。

05. D

在外部排序过程中输入/输出缓冲区就是排序的内存工作区, 例如进行 m 路平衡归并需要 m 个输入缓冲区和 1 个输出缓冲区, 用以存放参加归并的和归并完成的记录。在产生初始归并段时也可用作内部排序的工作区。它没有传送用户界面的消息的任务。

06. C

归并趟数 $= \lceil \log_k n \rceil$, 其中 k 表示归并的路数, n 表示元素个数, 当 $k = 4$ 、 $n = 64$ 时, 归并趟数恰好等于 3, 因此选取的归并路数至少是 4。

07. A

置换-选择排序是外部排序中生成初始归并段的方法, 用此方法得到的初始归并段的长度是不等长的, 其长度平均是传统等长初始归并段的 2 倍, 从而使得初始归并段数减少到原来的近二分之一。但是, 置换-选择排序不是一种完整的生成有序文件的外部排序算法。

08. D

设内存工作区 $w = 1$, 则文件 $\{1, 2, 3, 4, 5\}$ 产生 1 个有序段, 而文件 $\{5, 4, 3, 2, 1\}$ 产生 5 个有序段, 因此 m 与待排文件、内存工作区大小 w 和 n 都有关, 但不是选项 A、B、C 描述的直接关系。

09. C

在败者树中选取最小关键字的时间复杂度取决于败者树的高度, 所需时间为 $O(\log_2 k)$ 。

10. C

说法 I 正确, 是败者树的性质。在败者树的维护过程中, 会让胜利者一直调整到根结点, 说法 II 正确。以小根堆为例, 每次调整时, 先比较下一层的两个元素 (1 次), 找出较小值, 然后比较当前元素和下一层的较小元素 (1 次), 以决定是否向下交换位置, 说法 III 正确。堆在维护时, 可能会在中间某层停止 (若此处无须调整), 而不一定要走到叶结点, 说法 IV 错误。

11. B

最佳归并树在外部排序中的作用是设计 m 路归并排序的优化方案, 仿照构造哈夫曼树的方法, 以初始归并段的长度为权值, 构造具有最小带权路径长度的 m 叉哈夫曼树, 可以有效地减少归并过程中的读/写记录数, 加快外部排序的速度。

12. C

k 阶最佳归并树中只有度为 0 和 k 的结点。设结点总数为 n , 度为 0 的结点数为 n_0 , 度为 k 的结点数为 n_k , 则 $n_0 = m$ 、 $n - 1 = kn_k$, $n = n_0 + n_k$, 因此 $kn_k = m + n_k - 1$, 求得 $n_k = (m - 1)/(k - 1)$ 。

13. B

题中的三叉树为使 WPL 最小, 必须构造三叉哈夫曼树, 应满足 $(n_0 - 1) \% (3 - 1) = 0$ 的条件, 因